DynamAuction: A Reinforcement Learning Environment for Repeated Auctions with Dynamic Value

Eugénie Patard[1,*] **Corentin Pla**[1,2,3,*] **Benjamin Heymann**[1,2,†] **Patrick Loiseau**[1,†]

[1]INRIA, FairPlay Joint Team
[2] Criteo AI Lab
[3] CREST-ENSAE
Paris, France

`DynamAuction`

Abstract

A buyer who repeatedly participates in auctions for related items often faces a dynamic value problem: what they win today changes what tomorrow's items are worth. Despite the empirical importance of such effects, especially in display advertising, the algorithmic literature on repeated auctions has largely focused on the learning problem, by treating each opportunity as a contextual bandit, while sidestepping the planning problem induced by the dependence of current value on past wins. We introduce DynamAuction, a synthetic environment for single-buyer sequential bidding. The reward from each win depends on the buyer's history of previous wins, summarized through a carry-over state. The environment supports first- and second-price auctions, non-homogeneous Poisson arrivals, delayed feedback, saturation, and recency effects. We also introduce a new class of Model Predictive Control (MPC) algorithms. On our benchmark scenario, these MPC policies outperform off-the-shelf deep RL baselines (PPO, SAC, TD3) as well as a tuned shading baseline. We release DynamAuction to support RL research on repeated auctions.

1 Introduction

Motivation. In many real-world bidding settings, a buyer participates in a long sequence of auctions for related items, and the marginal value of winning the next one depends on what they have already won. The simplest instance is a buyer with single-unit demand: a firm that needs to acquire exactly one piece of equipment over a quarter through a series of auctions derives no additional value from a second win, so the value of the next opportunity drops to zero as soon as one auction has been won. More complex dynamics arise in display advertising. For example, showing the same banner to a user multiple times within a short period typically leads to diminishing returns, so the value of the next impression depends on the timing and outcomes of previous auctions.

Learning vs. planning in repeated auctions. Most existing work on repeated auctions focuses on learning how much to bid from contextual information and an unknown competition distribution [42, 1, 20]. The only planning component typically included is a global budget or ROI constraint, which couples bids over time through a Lagrangian or dual variable [20, 41, 7, 5]. Beyond this constraint,

*Equal contribution
†Equal supervision

40th Conference on Neural Information Processing Systems (NeurIPS 2026). Track on Evaluations and Datasets.

these approaches generally assume that the value of each opportunity depends only on its current context, without accounting for how past wins affect future values. In many applications, improving planning by accounting for these dynamics may be more beneficial than further improving learning under a static value model. Our work aims to study this question by introducing a reinforcement learning environment in which these dependencies can be explicitly modeled and evaluated.

Motivations for a synthetic environment. Evaluating bidding algorithms on real-world data presents several challenges. Logged auction data only reflect the decisions made by the bidding policy used to collect them, making it difficult to evaluate alternative strategies without additional assumptions. Online evaluation through A/B tests is the textbook alternative, but it is slow, costly, constrained by business and ethical risk, and does not let the researcher independently vary delay, saturation, recency, or interference. A simulator avoids these limitations by allowing each factor to be varied under controlled conditions. Although DYNAMAUCTION can be used with general reinforcement learning algorithms, our experiments focus on planning and on the value of accounting for past wins.

Contributions.

- We formalize a model of single-buyer sequential bidding in which the per-display reward is generated by a kernel conditioned on a *carry-over context* obtained by aggregating past wins through a *commutative monoid*. This framework accommodates delayed feedback, saturation and recency effects, exogenous interference, and user heterogeneity.
- We release DYNAMAUCTION, an open-source, Gymnasium-compatible implementation of this model, together with a default benchmark task and guidelines for users to define new tasks.
- We implement and evaluate a panel of baselines (constant bid, prediction-based bidding, greedy myopic bidding with shading, PPO, SAC, TD3) and introduce a class of Model Predictive Control algorithms. We show empirically that, on the benchmark scenario, this myopic-optimal MPC outperforms the deep-RL baselines and a tuned shading baseline.

Outline. Section 2 reviews the related literature and motivates the design of the environment. Section 3 presents the model, and Section 4 introduces DYNAMAUCTION. Section 5 describes our MPC approach. Section 6 presents the experiments, and Section 7 concludes. We use display advertising as the main running example throughout the paper. Accordingly, we refer to a won auction as a *display* and sometimes to its reward as a *conversion*.

2 Related work

2.1 Repeated auctions, attribution, and budget/ROI constraints

Much of the literature on repeated auctions studies a single bidder facing an unknown competition distribution, using bandit [32] or online learning methods. For example, [42, 1] consider repeated second-price auctions where the bidder learns the value of an item only after winning it. They develop bidding strategies with theoretical guarantees in stochastic and adversarial settings. Other works extend this framework to include global budget or ROI constraints, often using knapsack formulations [7, 5, 14, 13, 20, 19, 26]. This perspective has received considerable attention in the theoretical literature and has led to several important modeling contributions [4, 6, 17]. A recent survey is provided in [2].

A second line of work uses reinforcement learning (RL) to model sequential bidding as a Markov Decision Process (MDP). [11] formulate real-time bidding as an MDP whose state encodes the campaign's remaining budget and lifetime together with the current bid request, and learn a value-function approximation for the resulting bidding policy. [43] propose a model-free RL approach for budget-constrained bidding, while [44] treat bid optimization as a multivariable control problem for a set of campaigns. A more general RL framework with budget and KPI constraints is introduced in [25]. More recent work has questioned whether the current state contains enough information for effective bidding decisions. This has motivated generative approaches that model entire bidding trajectories rather than relying only on the current state [22, 35]. However, these approaches mainly

capture dependencies between bidding decisions through campaign-level constraints, such as budgets or ROI targets. The value of an impression is generally determined by its current context, without explicitly modeling how previous wins affect the value of future opportunities.

A third line of work focuses on *attributing* observed conversions back to the displays that caused them, and on bidding directly on incremental rather than observed outcomes. The general framing of computational advertising as a counterfactual learning problem goes back to [10]. On the bidding side, [18] show empirically that attribution-aware bidding outperforms last-touch baselines, and [9] and [8] develop causal models for repeated user interactions, the latter introducing the myopic-optimal valuation that we will reuse. [33] formalize the link between incrementality and attribution. [27] introduce the concept of repeated auctions with dynamic value and consider a setting simple enough to be solved with a differential equation, [28] leverage the analysis to build a no-regret learning algorithm based on plugin estimators, and [3] study incrementality bidding via reinforcement learning under mixed and delayed rewards. These works highlight the importance of accounting for past interactions, diminishing returns, and delayed outcomes. Our environment allows these effects to be modeled explicitly and studied under controlled conditions.

2.2 RL benchmarks, simulators, and datasets for auctions

A growing collection of open resources targets the bidding problem in online advertising. We list below those most relevant to our work.

Datasets. These contributions mostly cast the auction as a supervised learning problem. [34, 45] release the iPinYou dataset, the first publicly available large-scale RTB log, consisting of ≈ 35 GB of bid, impression, click, and conversion logs from a 2013 DSP competition. No environment for off-policy evaluation is provided. [18] release the Criteo Attribution Modeling for Bidding dataset, which contains 30 days of real advertising data with attribution labels. The dataset can be used to study attribution and incrementality, but does not provide an environment for simulating auction outcomes under different bidding strategies.

Simulators. [30] release AUCTIONGYM: a Python simulator of first- and second-price advertising auctions designed for the off-policy evaluation of bandit-style learning-to-bid algorithms. It provides configurable competition; however, the per-impression value is a function of the current context only.

Datasets paired with simulators. More recently, [39] release the simulated environment of the Alibaba/NeurIPS 2024 auto-bidding competition: it consists of a large dataset and an RL-ready state/action/reward interface designed for cost-per-action-constrained bidding. [31] release BAT, a benchmark built on the Avito platform, comprising 10 million first-price and 1 million Vickrey–Clarke–Groves (VCG) auction records collected over a month, together with a simulator and budget-pacing and cost-per-click (CPC) constrained baselines.

None of these resources is designed to exhibit the dependence of a single bidder's per-opportunity value on the history of past wins. DYNAMAUCTION is therefore complementary: AUCTIONGYM, BAT, and the AuctionNet environment are natural sandboxes for studying learning-to-bid and budget/ROI/Cost Per Auction (CPA)-constrained pacing; the iPinYou and Criteo datasets are appropriate for Click-Through Rate (CTR)/Conversion Rate (CVR) and attribution modeling on real logs; and DYNAMAUCTION is intended as a sandbox for studying planning under the dynamics induced by past wins on the value of future opportunities.

2.3 Model Predictive Control and model-based RL

Model Predictive Control (MPC) [12] uses a predictive model to optimize a sequence of actions over a finite horizon, executes only the first action, and replans at the next decision step. By evaluating candidate actions through model predictions rather than direct interactions with the environment, MPC is particularly appealing in settings where real-world experimentation is costly, provided that a sufficiently accurate model is available. MPC is an instance of a broader class of model-based approaches that use learned or known dynamics to guide decision-making. In model-based reinforcement learning, these approaches are commonly distinguished by how and when the model is used. In *decision-time planning*, the model is used at each decision step to evaluate candidate action sequences and select an action. PETS [16] is a representative example, as are the MPC algorithms proposed in Section 5. In *background planning*, the model is used during training to simulate

trajectories and improve a policy or value function. Unlike decision-time planning, these methods learn a policy using model-generated trajectories, rather than using the model to optimize actions at each decision step. MBPO [29] and Dreamer [24] follow this approach. MuZero [37] combines elements of both: it learns a compact latent dynamics model used for decision-time planning through Monte Carlo tree search, while training the model to support value and reward predictions rather than reconstructing the full environment. Model-based approaches have also been explored in real-time bidding, primarily to address budget allocation, pacing, and ROI or CPA constraints [11, 15]. In contrast, our setting focuses on a different source of temporal dependence: rather than campaign-level constraints, past auction wins influence the future marginal value of targeting the same user. The closest related work is repeated bidding with dynamic valuations [27]. Building on this perspective, DYNAMAUCTION provides a configurable benchmark for studying such sequential bidding decisions.

2.4 Benchmark Design Requirements

Based on the literature reviewed above, we identify three key properties for benchmarking sequential bidding with dynamic valuations. *Delayed feedback.* The reward from winning an auction may be observed only after a stochastic delay, which can be long. *Diminishing returns.* The marginal value of winning an auction may decrease as the bidder accumulates wins for the same user, reflecting saturation effects. *Recency effects.* The marginal value of a win may also depend on the time elapsed since previous wins, rather than only on their number. Beyond these three properties, the environment should allow for heterogeneous users, exogenous events affecting user behavior, different opponent bid distributions and auction formats, and configurable auction arrival processes. We do not consider campaign-level budget or ROI constraints, which have been extensively studied and are separate from the user-level dynamics considered here. They could nevertheless be incorporated in future extensions.

3 Model: sequential bidding with carry-over value

3.1 Buyer, opportunities, and arrival process

We consider a single buyer interacting over a finite, possibly random, time horizon $[0, T]$ where $T > 0$, with a user u drawn from a set $\mathcal{U}$ according to a probability measure ρ. A *bidding opportunity* for u is an event at which the buyer is offered the chance to bid in an auction for an item[3] described by a feature vector $x \in \mathcal{X}$, where $\mathcal{X}$ is a measurable feature space. Bidding opportunities for a fixed user u arrive at random times $0 \le \theta_1 < \theta_2 < \cdots$ generated by a non-homogeneous Poisson process on $[0, T]$ with intensity $\mu : [0, T] \to \mathbb{R}_{\ge 0}$, and item feature $X_j \in \mathcal{X}$ is drawn from a conditional distribution $\nu(\cdot \mid u, \theta_j)$. The buyer observes X_j, places a bid $b_j \in \mathbb{R}_{\ge 0}$, and competes against an opponent's bid $B_j \in \mathbb{R}_{\ge 0}$ drawn independently from a competition distribution F.[4] The auction is resolved by an allocation rule $\mathcal{W}(b, B) \in \{0, 1\}$ (winning indicator) and a payment rule $p(b, B) \in \mathbb{R}_{\ge 0}$. For instance, the first-price rule corresponds to $\mathcal{W}(b, B) = \mathbf{1}\{b > B\},\ p(b, B) = b\,\mathcal{W}(b, B)$ and the second-price rule corresponds to $\mathcal{W}(b, B) = \mathbf{1}\{b > B\},\ p(b, B) = B\,\mathcal{W}(b, B)$. We let $W_j \in \{0, 1\}$ denote whether opportunity j is won. We write $t_1 < t_2 < \cdots$ for the corresponding *win times*, that is, the opportunity times θ_j such that $W_j = 1$. Finally, x_i denotes the feature of the item won at time t_i. User heterogeneity is represented through the dependence of the model primitives on u: the arrival intensity $\mu(\cdot\,|u)$, the item-feature distribution $\nu(\cdot \mid u, \theta)$, the law of the exogenous process Π_u, and the reward kernel $\mathcal{K}(u, \cdot)$ may all depend on u (see below for the definition of these terms). The population measure ρ on $\mathcal{U}$ then determines how this heterogeneity aggregates into the buyer's objective (4). For $t \in [0, T]$, the set of win indices up to time t is $\mathcal{I}_u(t) = \{i : t_i \le t\}$, and we write $\mathcal{I}_u(t^-) = \{i : t_i < t\}$ for the wins strictly before t.

3.2 Carry-over context as a commutative monoid

To capture the dependence of the buyer's per-opportunity value on past wins, we summarize the user's display history into a single state. We fix a (measurable) space[5] $\mathcal{S}$ together with

[3] In the context of RTB auctions, this is a placement for an online banner on a publisher website.

[4] It is standard to aggregate all the competition into a single price-to-beat.

[5] In what follows, we omit most of the measurability assumptions.

a commutative monoid[6] structure $(\mathcal{S}, \oplus, \mathbf{0})$. The binary operation $\oplus : \mathcal{S} \times \mathcal{S} \to \mathcal{S}$ is associative and commutative, and $\mathbf{0} \in \mathcal{S}$ is its identity element. We further fix a *decay map* $\gamma : \mathcal{X} \times [0,T] \times [0,T] \to \mathcal{S}, \quad (x, t', t) \mapsto \gamma(x, t', t)$, intended to be evaluated only at $t' \leq t$, and which encodes the residual contribution at time t of a display with feature x that was shown at the earlier time t'. For a fixed user $u \in \mathcal{U}$ with realized win sequence $(t_i, x_i)_{i \in \mathcal{I}_u(t)}$ at time $t \in [0,T]$, we define the *carry-over context* at time t as

$$S_u(t) \;=\; \bigoplus_{i \in \mathcal{I}_u(t)} \gamma(x_i, t_i, t) \;\in\; \mathcal{S}, \tag{1}$$

with the empty sum convention $S_u(t) = \mathbf{0}$ when $\mathcal{I}_u(t) = \emptyset$. Since $\oplus$ is associative and commutative, the value of the sum does not depend on the order in which past wins are aggregated. By construction, S_u is càdlàg in t: it jumps at each win time t_i and evolves smoothly between wins, through the dependence of γ on its third argument. Familiar instances of $(\mathcal{S}, \oplus, \mathbf{0})$ that we use in the experiments include $(\mathbb{R}_{\geq 0}, +, 0)$ for additive counters, $([0,1], \times, 1)$ for multiplicatively decreasing marginal effects, and $([0, +\infty], \min, +\infty)$ for the time since the most recent qualifying win. For the recency context, a qualifying display contributes $\gamma(x, t', t) = t - t'$, while a nonqualifying display contributes $+\infty$. Taking the minimum gives the time since the most recent qualifying win, or $+\infty$ if there has been none. A different choice is to retain the full display history. We use $(\mathsf{H}, \cup, \emptyset)$, where H is the collection of finite sets of display–time pairs (x_i, t_i). Each display contributes $\gamma(x, t', t) = \{(x, t')\}$, and these contributions are combined by set union. The timestamps allow us to recover the chronological order of the displays.

3.3 Reward kernel

We define a *reward kernel* that describes how the expected reward of a display is distributed over time. It is modeled as a positive measure over conversion delays, parametrized by the user, the displayed item, the carry-over context at the time of display, and an exogenous information process (introduced in Section 3.4). Formally, we define a measurable map

$$\mathcal{K} : \mathcal{U} \times \mathcal{X} \times \mathcal{S} \times \mathcal{Y} \times [0,T] \;\to\; \mathcal{M}_+\big([0,\infty)\big),$$

where $\mathcal{Y}$ is the value space of the exogenous information process and $\mathcal{M}_+\big([0,\infty)\big)$ denotes the set of positive Borel measures on $[0,\infty)$. For a display occurring at time t, the kernel specifies the expected reward associated with different delays after the display. For any Borel measurable set $A \subseteq [0,\infty)$ of such delays, we write

$$\mathcal{K}(u, x, s, y, t)(A) \;=\; \kappa(u, x, s, y, t)\delta_0(A) \;+\; \int_A \lambda(u, x, s, y, t; \tau)\, \mathrm{d}\tau, \tag{2}$$

where the Dirac component κ models the *instantaneous* part of the reward, while the absolutely continuous component λ models the *delayed* part as a density over the delay $\tau \geq 0$ since the display. For a single won display at time t_i on user u with feature x_i, the expected reward attributable to this display and generated over a time window $[t_i, t_i + h]$ for $h \geq 0$ is

$$r_i(h) \;=\; \kappa\big(u, x_i, S_u(t_i^-), y_u(t_i), t_i\big) \;+\; \int_0^h \lambda\big(u, x_i, S_u(t_i^-), y_u(t_i), t_i;\, \tau\big)\, \mathrm{d}\tau, \tag{3}$$

where $S_u(t_i^-)$ denotes the left-limit of the carry-over context at t_i, i.e., the carry-over context evaluated just *before* the i-th win is incorporated into it.[7] The buyer's expected cumulative reward over the horizon $[0,T]$ is then obtained by integrating per-display contributions over all wins occurring before T against the population measure ρ,

$$\mathcal{R}(T) \;=\; \int_{\mathcal{U}} \mathbb{E}\left[\sum_{i \in \mathcal{I}_u(T)} r_i(T - t_i) \right] \rho(\mathrm{d}u), \tag{4}$$

where the expectation is over the arrival process, the item-feature distribution, the competition, the exogenous process, and any randomness in the bidding policy, all conditional on the user u. The expected cumulative payment is $\mathcal{C}(T) \;=\; \int_{\mathcal{U}} \mathbb{E}\Big[\sum_{j:\theta_j \leq T} p(b_j, B_j) \Big]\, \rho(\mathrm{d}u)$, and the buyer's net cumulative profit $\mathcal{R}(T) - \mathcal{C}(T)$ is what they seek to maximize.

[6] A monoid is a set equipped with an associative binary operation and an identity element.

[7] The left-limit convention ensures that a winning bid is rewarded against the context that justified placing it, rather than against a context that already includes the consequence of the win itself.

3.4 Exogenous information and information available to the buyer

Exogenous information process. For each user $u \in \mathcal{U}$, we allow exogenous events that affect the value of future displays but are independent of the buyer's actions. These events occur at times $0 \le \tau_1^u < \tau_2^u < \cdots$ and have marks $e_\ell^u \in \mathcal{E}$. We treat the marked point process $\Pi_u = (e_\ell^u, \tau_\ell^u)_{\ell \ge 1}$ as a primitive of the model and assume it is independent of the buyer's bidding policy. We then summarize the past of this process into a càdlàg *exogenous information process*

$$y_u : [0, T] \to \mathcal{Y}, \qquad y_u(t) \;=\; \Psi\big(\{(e_\ell^u, \tau_\ell^u) : \tau_\ell^u \le t\}\big), \tag{5}$$

The resulting state $y_u(t)$ affects the reward generated by a display at time t but evolves independently of the buyer's actions.

Information available to the buyer. Let $\phi : \mathcal{S} \times [0, \infty) \to \mathcal{O}_S$ be a measurable observation map, where $\mathcal{O}_S$ is the space of reported carry-over summaries. Full observation corresponds to $\mathcal{O}_S = \mathcal{S}$ and $\phi(s, t) = s$. At an opportunity time θ_j on user u, we assume the buyer observes the candidate item feature x_j, the user identity u, the carry-over summary $\phi(S_u(\theta_j^-), \theta_j)$, the exogenous process value $y_u(\theta_j)$, and the time θ_j itself. We collect this information in the *observable state*.

$$Z_j = \big(u, X_j, \phi(S_u(\theta_j^-), \theta_j), y_u(\theta_j), \theta_j\big) \in \mathcal{U} \times \mathcal{X} \times \mathcal{O}_S \times \mathcal{Y} \times [0, \infty).$$

After each auction, the buyer observes the win indicator and payment. The scenario specifies whether the competing bid B_j is also disclosed and how delayed rewards are reported. Policies may use current and past observations, past bids, feedback already received, and private randomization. The observation Z_j need not be Markov. Writing $b_j = \pi(Z_j)$ restricts attention to policies using only the current observation. Figure 1 summarizes the interaction process.

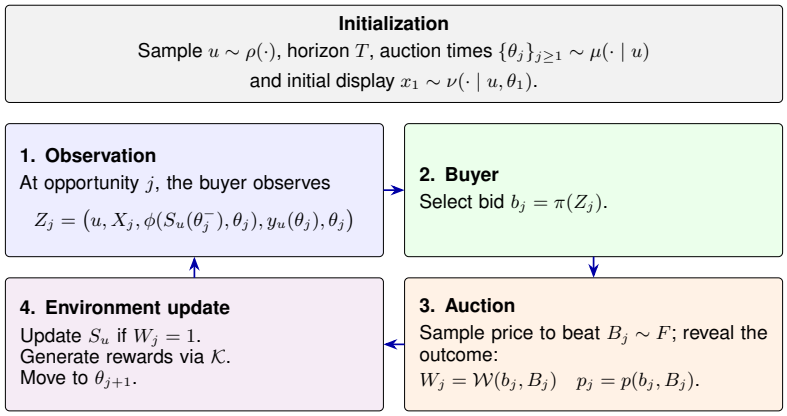


Figure 1: Overview of DYNAMAUCTION, illustrating the interaction between the bidding agent and the environment, the dependence of future values on past wins, and the timing of rewards.

4 DynamAuction

DYNAMAUCTION implements the model introduced in the previous section as a Gymnasium environment. It allows researchers to evaluate bidding strategies under different auction dynamics through configurable parameters and provides a set of baseline policies for comparison. Additional information is provided in Appendix A.

Scenario configuration. In DYNAMAUCTION, a scenario defines the model parameters, including auction mechanism, reward kernel, carry-over context, competition distribution, campaign horizon, and arrival process. Scenarios can be saved and reused across experiments, making it easy to compare bidding strategies under the same conditions. Table 2 presents the predefined scenarios available in DYNAMAUCTION.

Gymnasium interface. DYNAMAUCTION follows the Gymnasium API [40] and can be used with RL libraries such as Stable-Baselines3 [36]. We provide three wrappers to adapt the environment for RL training (Table 3). The recommended order is: (i) *oracle masking*, which removes expected-reward information that would not be available to a real bidder; (ii) *observation flattening*, which converts the observations into a single vector for standard neural network policies; and (iii) *bid clipping*, which limits bids to $[0, b_{\max}]$ during training for stable exploration.

5 Model Predictive Control with Myopic Valuations

5.1 Myopic-optimal valuation

The planning problem introduced in Section 3 can, in principle, be solved using dynamic programming. The optimal bid at time t must account not only for the reward obtained by winning the current auction, but also for its impact on future rewards. As shown in [9, Theorem 3.1], this trade-off can be characterized using the Bellman value function $V^\star$, which represents the maximum expected cumulative reward achievable from a given state z. However, estimating $V^\star$ is challenging. The state space is high-dimensional, online exploration is costly, offline counterfactual estimation suffers from high variance, and the environment is non-stationary. Instead, we build on the notion of *myopic-optimal valuation* introduced in [8]. The myopic-optimal value V_0 is defined as the optimal bid for the current display under two assumptions: (i) the bidder cannot participate in any future auction on the same user, and (ii) the current auction is second-price. In our model, V_0 corresponds to the expected reward generated by the current display, obtained by integrating Equation (3) over the full delay support. It can be estimated using a fixed continuation policy that places no further bids on the same user, thereby isolating the reward of the current display from future bidding decisions. Additional algorithmic details are provided in Appendix B.

5.2 Planning with Myopic-optimal Valuations

At each decision step, our Model Predictive Control (MPC) approach compares candidate bids through Monte Carlo rollouts over a finite planning horizon. It submits the bid with the highest estimated net reward and replans at the next auction. An estimator $\widehat{V}_0$ of the myopic-optimal value provides the scale for both the candidate bids and a fixed continuation policy. Rollout scores are computed from simulated conversions and payments. Thus, the optimization concerns a single multiplier for the current bid, while accounting for its consequences for future opportunities.

Candidate bid. We express each candidate bid as a multiple of the estimated myopic-optimal value:

$$b(\alpha; Z) = \alpha \widehat{V}_0(Z), \tag{6}$$

where $\alpha \in \mathcal{A}_0 \subset \mathbb{R}_{\geq 0}$ is selected from a finite set of candidate multipliers. In a second-price auction, $\alpha = 1$ bids the estimated myopic value, while $\alpha < 1$ allows the planner to reduce the bid when winning the current auction has an adverse effect on future opportunities.

Simulating future auctions. At auction i, at time θ_i, we generate M rollouts from copies of the current simulator state, retaining past wins and pending conversions. The current observation Z_i is held fixed, while the remaining campaign duration and future events are resampled according to the scenario model. Let L_m denote the last auction time in the complete sampled campaign, including the current auction. The planning window contains the auction indices $\mathcal{N}_H^{(m)}$ with $\theta_i \leq \theta_k^{(m)} \leq \min(\theta_i + H, L_m)$, where H is measured in time units. The current auction uses $b(\alpha; Z_i)$, and all subsequent auctions in this window use $b(\alpha_{\text{cont}}; Z_k^{(m)})$, with α_{cont} fixed across candidates. The continuation rule is fixed, but its bids depend on the observations reached in each rollout. All candidates are evaluated on the same sampled campaigns, using common random numbers for competing bids, display features, and conversions, with draws aligned by auction to reduce the variance of score differences.

Evaluating simulated trajectories. For each simulated win, the display's conversions are sampled using its pre-win carry-over context. Let $C_{k,L_m}^{(m)}$ denote the total value of these conversions occurring no later than L_m. The contribution of auction k to the rollout score is

$$g_k^{(m)}(\alpha) = \mathcal{W}\left(b_k^{(m)}, B_k^{(m)}\right) C_{k,L_m}^{(m)} - p\left(b_k^{(m)}, B_k^{(m)}\right), \tag{7}$$

where $b_k^{(m)} = b(\alpha_k^{(m)}; Z_k^{(m)})$, $\alpha_i^{(m)} = \alpha$, and $\alpha_k^{(m)} = \alpha_{\text{cont}}$ for $k > i$. The allocation and payment rules $\mathcal{W}$ and p are those defined in Section 3, with zero payment after a loss. For a lost auction, we set $C_{k,L_m}^{(m)} = 0$.

The planning horizon limits the auctions for which decisions are simulated, not the delays of their conversions. A display won within the window may therefore contribute conversions after $\theta_i + H$, provided they occur before L_m, consistently with the environment's reward reporting. Contributions from displays won before auction i are omitted from the score because they do not depend on the candidate. The effects of simulated wins on subsequent opportunities are captured through the evolution of the carry-over context.

For each candidate, we average these returns over the M rollouts:

$$\widehat{Q}_i(\alpha) = \frac{1}{M} \sum_{m=1}^{M} \sum_{k \in \mathcal{N}_H^{(m)}} g_k^{(m)}(\alpha). \tag{8}$$

Selecting the bid. We select the multiplier with the highest estimated net reward, breaking ties in favor of the smallest multiplier:

$$\alpha_i^\star = \min \arg\max_{\alpha \in \mathcal{A}_0} \widehat{Q}_i(\alpha), \qquad b_i = b(\alpha_i^\star; Z_i). \tag{9}$$

After submitting b_i, the bidder receives the auction feedback and repeats the optimization at the next opportunity. Pseudocode and implementation details are provided in Appendix B.

6 Experiments

Algorithms. To illustrate the use of DynamAuction as a benchmark for bidding with dynamic valuations, we evaluate eight bidding policies, ranging from simple baselines to model-free reinforcement learning as well as our MPC approach. The policies are the following:

- Constant: Bids a fixed amount b^* selected by grid search.
- FiPla, introduced by [8] that bids the true myopic-optimal value $V_0(Z)$.
- Greedy Myopic, that estimates the marginal return of winning the current auction by comparing Monte Carlo rollouts that force a win or a loss, followed by a zero-bid continuation policy. The estimated difference is then used to determine the bid.
- Shading uses the same estimator as Greedy Myopic but scales the resulting bid by a constant multiplier α^* optimized through grid search.
- MPC, introduced in Section 5.
- Deep RL: We consider three model-free algorithms implemented with Stable-Baselines3 [36]: PPO [38], SAC [23], and TD3 [21].

All policies are evaluated on the same 1,000 test campaigns to ensure consistent comparisons. For each RL algorithm, we select hyperparameters using 200 validation campaigns and train three models with different random seeds. We report the average performance across these three runs. Further experimental details are provided in Appendix C.

Scenario. We choose a scenario in which current bidding decisions have lasting effects on future rewards, making planning a central aspect of the task. This can be modelled by advertising fatigue: a user repeatedly exposed to the same advertisement over a short period may become less receptive to subsequent impressions, with an effect that persists throughout the campaign. The bidder must therefore balance the expected reward of winning the current auction against its effect on the value of subsequent opportunities, making the timing and spacing of wins relevant to campaign performance. This is a deliberate modeling choice for our benchmark. Users of DynamAuction are free to specify alternative carry-over dynamics, reward kernels, and scenario parameters to study different mechanisms or applications.

Formally, all main experiments use the `survival-second-price-exponential` scenario, a second-price (Vickrey) auction environment with a random campaign duration $T \sim \text{Exp}(1/\eta)$,

where η is the mean duration. We consider $\eta = 50$, the corresponding test campaigns contain 47.7 auctions on average. Auction opportunities arrive according to a Poisson process of rate 1, and the competing bid is $B \sim \mathrm{LogNormal}(-1, 1)$. A user feature vector $u \sim \mathcal{N}(0, I_3)$ is drawn once per campaign, and a display feature vector $x \sim \mathcal{N}(0, I_3)$ is drawn at each auction. Performance is reported as the mean episodic return over 1 000 test campaigns. The remaining parameters are listed in Tables 5 and 6.

Carry-over context and observations. To instantiate the commutative-monoid framework introduced in Section 3.2, we take the carry-over context to be the list of past wins:

$$S_u(t^-) = \{t_i < t\}. \tag{10}$$

At time t, the buyer only receives a scalar fatigue factor $\phi(S_u(t^-), t)$ computed from this history. The buyer therefore does not have access to the full carry-over context, making the scenario partially observable. For $N \geq 1$ past wins at times $t_1 < \cdots < t_N < t$, define

$$\phi(S_u(t^-), t) = \left[\prod_{i=2}^{N} \left(1 - ce^{-\beta(t_i - t_{i-1})}\right)\right] \left(1 - ce^{-\beta(t - t_N)}\right), \qquad c = 0.9, \quad \beta = 1. \tag{11}$$

We use the empty-product convention and set $\phi(\varnothing, t) = 1$ before the first win. Each completed interval between consecutive wins contributes a fixed multiplicative penalty. Closely spaced wins cause a persistent reduction in subsequent conversion propensity within the campaign. If the current auction at time t is won, its timestamp is added to the history after evaluating the current display's reward kernel, so it affects later displays through their carry-over context. The observation fields are described in Table 4.

Delayed reward. There is no exogenous signal ($y_u \equiv 0$) and no instantaneous reward component ($\kappa = 0$). For delay $\tau > 0$, the reward kernel is

$$\mathcal{K}(u, x, S, 0, t)(\mathrm{d}\tau) = \mathrm{clip}\big(\phi(S, t)(\mathbf{1}^\top u)(\mathbf{1}^\top x), 0, 1\big)\ e^{-\tau}\,\mathrm{d}\tau,$$

Conditional on the realized campaign end T, the expected conversion reward attributable to a display won at $t_i < T$ is

$$V_i^{\mathrm{oracle}} = \mathrm{clip}\big(\phi(S, t)(\mathbf{1}^\top u)(\mathbf{1}^\top x), 0, 1\big)\left(1 - e^{-(T - t_i)}\right).$$

We call this quantity the oracle value because it depends on the realized end time T, which is unknown to the bidder. The oracle value is available to FiPla but hidden from the other agents (Table 3).

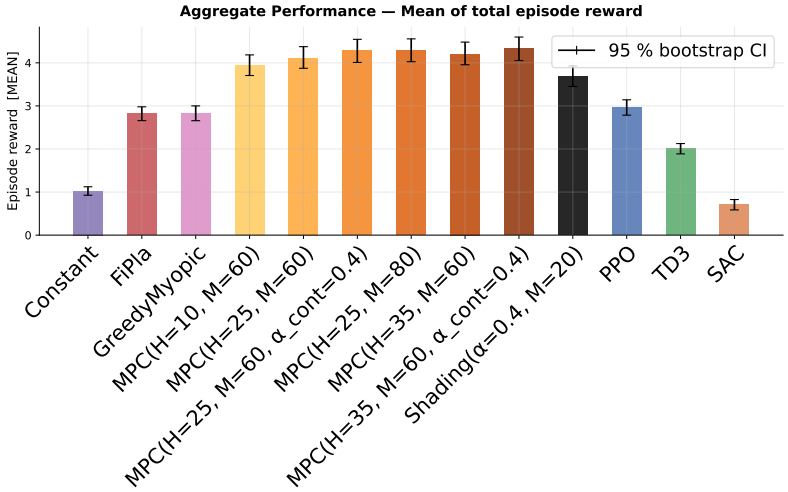


Figure 2: Mean episode reward per policy at $\eta = 50$ over the 1 000 test campaigns. Error bars are 95% bootstrap confidence intervals. RL agents: mean of their three trainings per campaign.

Table 1: Mean episode reward ($\pm$ 95% CI) over the 1 000 test campaigns, and paired difference with Shading($\alpha = 0.4$, $M = 20$): its CI is computed from the per-campaign differences, which removes the variability between campaigns. RL agents: mean of their three trainings per campaign. Settings in Appendix C.

Policy	Reward	Δ vs Shading
MPC($H = 35$, $M = 60$, $\alpha_{\text{cont}} = 0.4$)	4.328 ± 0.275	$+0.65 \pm 0.13$
MPC($H = 25$, $M = 60$, $\alpha_{\text{cont}} = 0.4$)	4.285 ± 0.268	$+0.61 \pm 0.12$
MPC($H = 25$, $M = 80$)	4.282 ± 0.268	$+0.60 \pm 0.13$
MPC($H = 35$, $M = 60$)	4.206 ± 0.269	$+0.53 \pm 0.13$
MPC($H = 25$, $M = 60$)	4.112 ± 0.257	$+0.43 \pm 0.11$
MPC($H = 10$, $M = 60$)	3.938 ± 0.238	$+0.26 \pm 0.11$
Shading($\alpha = 0.4$, $M = 20$)	3.680 ± 0.234	—
PPO (mean of 3 trainings)	2.965 ± 0.182	-0.71 ± 0.12
GreedyMyopic	2.826 ± 0.167	-0.85 ± 0.16
FiPla	2.821 ± 0.160	-0.86 ± 0.16
TD3 (mean of 3 trainings)	2.006 ± 0.121	-1.67 ± 0.15
Constant($b = 0.11$)	1.026 ± 0.099	-2.65 ± 0.19
SAC (mean of 3 trainings)	0.707 ± 0.119	-2.97 ± 0.24

Results. Table 1 and Figure 2 report the performance of the different policies. MPC outperforms the tuned Shading baseline when the planning horizon is sufficiently long ($H \geq 25$). The best configuration, MPC($H = 35$, $M = 60$, $\alpha_{\text{cont}} = 0.4$), achieves a mean episodic return of 4.33, compared with 3.68 for Shading and 2.82 for FiPla. The gain over Shading is $+0.65 \pm 0.13$. With a shorter planning horizon ($H = 10$), the improvement is smaller but remains positive ($+0.26 \pm 0.11$). Using a shaded continuation policy ($\alpha_{\text{cont}} = 0.4$) also improves MPC performance, although the gains are smaller. The deep RL algorithms perform worse than Shading on average. Their performance is also sensitive to training: PPO shows substantial variability across seeds, one of the three TD3 runs converges to a policy that never bids, and SAC wins considerably more auctions than FiPla but achieves a lower return. Additional experiments are reported in Appendix D.

7 Conclusion

We introduced a generic model for describing dynamic value in repeated auctions, along with DynamAuction, an RL environment for studying sequential bidding. The environment is challenging for deep reinforcement learning methods while accommodating settings that can be studied analytically, as in [27]. On our benchmark, which illustrates the motivation for planning in this setting, simple shading and model-predictive control policies outperform the tested deep RL methods. Our design focuses on isolating the effects of dynamic value. Accordingly, we use synthetic data and leave budget and ROI constraints outside the scope of this work.

Acknowledgments and Disclosure of Funding

This work was supported in part by the French National Research Agency (ANR) through grants ANR-20-CE23-0007 and ANR-23-CE23-0002, and through the PEPR IA FOUNDRY project (ANR-23-PEIA-0003). It was also supported by Hi! PARIS and the ANR/France 2030 program (ANR-23-IACL-0005).

References

[1] J. Achddou, O. Cappé, and A. Garivier. Efficient algorithms for stochastic repeated second-price auctions. In *Algorithmic Learning Theory*, pages 99–150. PMLR, 2021.

[2] G. Aggarwal, A. Badanidiyuru, S. R. Balseiro, K. Bhawalkar, Y. Deng, Z. Feng, G. Goel, C. Liaw, H. Lu, M. Mahdian, J. Mao, A. Mehta, V. Mirrokni, R. P. Leme, A. Perlroth, G. Piliouras, J. Schneider, A. Schvartzman, B. Sivan, K. Spendlove, Y. Teng, D. Wang, H. Zhang, M. Zhao, W. Zhu, and S. Zuo. Auto-bidding and auctions in online advertising: A survey. *SIGecom Exch.*, 22(1):159–183, Oct. 2024.

[3] A. Badanidiyuru Varadaraja, Z. Feng, T. Li, and H. Xu. Incrementality bidding via reinforcement learning under mixed and delayed rewards. *Advances in Neural Information Processing Systems*, 35:2142–2153, 2022.

[4] S. Balseiro, A. Kim, M. Mahdian, and V. Mirrokni. Budget management strategies in repeated auctions. In *Proceedings of the 26th International Conference on World Wide Web*, WWW '17, page 15–23, Republic and Canton of Geneva, CHE, 2017. International World Wide Web Conferences Steering Committee.

[5] S. Balseiro, H. Lu, and V. Mirrokni. Dual mirror descent for online allocation problems. In *International Conference on Machine Learning*, pages 613–628. PMLR, 2020.

[6] S. R. Balseiro, O. Besbes, and G. Y. Weintraub. Repeated auctions with budgets in ad exchanges: Approximations and design. *Management Science*, 61(4):864–884, 2015.

[7] S. R. Balseiro and Y. Gur. Learning in repeated auctions with budgets: Regret minimization and equilibrium. In *Proceedings of the 2017 ACM Conference on Economics and Computation*, EC '17, page 609, New York, NY, USA, 2017. Association for Computing Machinery.

[8] M. Bompaire, A. Désir, and B. Heymann. Fixed point label attribution for real-time bidding. *Manufacturing & Service Operations Management*, 26(3):1043–1061, 2024.

[9] M. Bompaire, A. Gilotte, and B. Heymann. Causal models for real time bidding with repeated user interactions. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*, KDD '21, page 75–85, New York, NY, USA, 2021. Association for Computing Machinery.

[10] L. Bottou, J. Peters, J. Quiñonero-Candela, D. X. Charles, D. M. Chickering, E. Portugaly, D. Ray, P. Simard, and E. Snelson. Counterfactual reasoning and learning systems: The example of computational advertising. *Journal of Machine Learning Research*, 14(11), 2013.

[11] H. Cai, K. Ren, W. Zhang, K. Malialis, J. Wang, Y. Yu, and D. Guo. Real-time bidding by reinforcement learning in display advertising. In *Proceedings of the Tenth ACM International Conference on Web Search and Data Mining*, WSDM '17, page 661–670, New York, NY, USA, 2017. Association for Computing Machinery.

[12] E. F. Camacho and C. Bordons. *Model Predictive Control*. Advanced Textbooks in Control and Signal Processing. Springer London, 2 edition, 2013.

[13] M. Castiglioni, A. Celli, and C. Kroer. Online learning under budget and roi constraints via weak adaptivity. In *Proceedings of the 41st International Conference on Machine Learning*, ICML'24. JMLR.org, 2024.

[14] M. Castiglioni, A. Celli, A. Marchesi, G. Romano, and N. Gatti. A unifying framework for online optimization with long-term constraints. *Advances in Neural Information Processing Systems*, 35:33589–33602, 2022.

[15] S. Chen, Q. Xu, L. Zhang, Y. Jin, W. Li, and L. Mo. Model-based reinforcement learning for auto-bidding in display advertising. In *Proceedings of the 2023 International Conference on Autonomous Agents and Multiagent Systems*, AAMAS '23, page 1560–1568, Richland, SC, 2023. International Foundation for Autonomous Agents and Multiagent Systems.

[16] K. Chua, R. Calandra, R. McAllister, and S. Levine. Deep reinforcement learning in a handful of trials using probabilistic dynamics models. In *Advances in Neural Information Processing Systems*, volume 31, 2018.

[17] V. Conitzer, C. Kroer, E. Sodomka, and N. E. Stier-Moses. Multiplicative pacing equilibria in auction markets. *Operations Research*, 70(2):963–989, 2022.

[18] Diemert Eustache, Meynet Julien, P. Galland, and D. Lefortier. Attribution modeling increases efficiency of bidding in display advertising. In *Proceedings of the AdKDD and TargetAd Workshop, KDD, Halifax, NS, Canada, August, 14, 2017*, page To appear. ACM, 2017.

[19] Z. Feng, S. Padmanabhan, and D. Wang. Online bidding algorithms for return-on-spend constrained advertisers. In *Proceedings of the ACM Web Conference 2023*, WWW '23, page 3550–3560, New York, NY, USA, 2023. Association for Computing Machinery.

[20] A. Flajolet and P. Jaillet. Real-time bidding with side information. In I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017.

[21] S. Fujimoto, H. van Hoof, and D. Meger. Addressing function approximation error in actor-critic methods, 2018.

[22] J. Guo, Y. Huo, Z. Zhang, T. Wang, C. Yu, J. Xu, B. Zheng, and Y. Zhang. Generative auto-bidding via conditional diffusion modeling. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 5038–5049, 2024.

[23] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor, 2018.

[24] D. Hafner, T. Lillicrap, J. Ba, and M. Norouzi. Dream to control: Learning behaviors by latent imagination. In *International Conference on Learning Representations*, 2020.

[25] Y. He, X. Chen, D. Wu, J. Pan, Q. Tan, C. Yu, J. Xu, and X. Zhu. A unified solution to constrained bidding in online display advertising. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, KDD '21, page 2993–3001, New York, NY, USA, 2021. Association for Computing Machinery.

[26] B. Heymann. Cost per action constrained auctions. In *Proceedings of the 14th Workshop on the Economics of Networks, Systems and Computation*, pages 1–8, 2019.

[27] B. Heymann, A. Gilotte, and R. Chan-Renous. Repeated bidding with dynamic value. In *International Conference on Web and Internet Economics*, pages 548–563. Springer, 2024.

[28] B. Heymann and O. Sakhi. Learning to bid in repeated second-price auctions with dynamic values and aggregated feedback. *to be published at Neurips 2026*, 2026.

[29] M. Janner, J. Fu, M. Zhang, and S. Levine. When to trust your model: Model-based policy optimization. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

[30] O. Jeunen, S. Murphy, and B. Allison. Off-policy learning-to-bid with auctiongym. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 4219–4228, 2023.

[31] A. Khirianova, E. Solodneva, A. Pudovikov, S. Osokin, E. Samosvat, Y. Dorn, A. Ledovsky, and Y. Zenkova. Bat: Benchmark for auto-bidding task. In *Proceedings of the ACM on Web Conference 2025*, WWW '25, page 2657–2667, New York, NY, USA, 2025. Association for Computing Machinery.

[32] T. Lattimore and C. Szepesvári. *Bandit algorithms*. Cambridge University Press, 2020.

[33] R. Lewis and J. Wong. Incrementality bidding and attribution. *arXiv preprint arXiv:2208.12809*, 2022.

[34] H. Liao, L. Peng, Z. Liu, and X. Shen. ipinyou global rtb bidding algorithm competition dataset. In *Proceedings of the Eighth International Workshop on Data Mining for Online Advertising*, ADKDD'14, page 1–6, New York, NY, USA, 2014. Association for Computing Machinery.

[35] Y. Peng, W. Shu, J. Sun, Y. Zeng, J. Pang, W. Bai, Y. Bai, X. Liu, and P. Jiang. Expert-guided diffusion planner for auto-bidding. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management*, CIKM '25, page 5963–5970, New York, NY, USA, 2025. Association for Computing Machinery.

[36] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann. Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268):1–8, 2021.

[37] J. Schrittwieser, I. Antonoglou, T. Hubert, K. Simonyan, L. Sifre, S. Schmitt, A. Guez, E. Lockhart, D. Hassabis, T. Graepel, T. Lillicrap, and D. Silver. Mastering atari, go, chess and shogi by planning with a learned model. *Nature*, 588(7839):604–609, Dec. 2020.

[38] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms, 2017.

[39] K. Su, Y. Huo, Z. Zhang, S. Dou, C. Yu, J. Xu, Z. Lu, and B. Zheng. Auctionnet: A novel benchmark for decision-making in large-scale games. *Advances in Neural Information Processing Systems*, 37:94428–94452, 2024.

[40] M. Towers, A. Kwiatkowski, J. Terry, J. U. Balis, G. De Cola, T. Deleu, M. Goulão, A. Kallinteris, M. Krimmel, A. KG, et al. Gymnasium: A standard interface for reinforcement learning environments. *arXiv preprint arXiv:2407.17032*, 2024.

[41] Q. Wang, Z. Yang, X. Deng, and Y. Kong. Learning to bid in repeated first-price auctions with budgets. In *Proceedings of the 40th International Conference on Machine Learning*, ICML'23. JMLR.org, 2023.

[42] J. Weed, V. Perchet, and P. Rigollet. Online learning in repeated auctions. In *Conference on Learning Theory*, pages 1562–1583. PMLR, 2016.

[43] D. Wu, X. Chen, X. Yang, H. Wang, Q. Tan, X. Zhang, J. Xu, and K. Gai. Budget constrained bidding by model-free reinforcement learning in display advertising. In *Proceedings of the 27th ACM International Conference on Information and Knowledge Management*, CIKM '18, page 1443–1451, New York, NY, USA, 2018. Association for Computing Machinery.

[44] X. Yang, Y. Li, H. Wang, D. Wu, Q. Tan, J. Xu, and K. Gai. Bid optimization by multivariable control in display advertising. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, KDD '19, page 1966–1974. ACM, 2019.

[45] W. Zhang, S. Yuan, J. Wang, and X. Shen. Real-time bidding benchmarking with ipinyou dataset. *arXiv preprint arXiv:1407.7073*, 2014.

Appendix

A Environment implementation and configuration **15**
A.1 Gymnasium Interface and Wrappers 15
A.2 Observation and Action Spaces 15
A.3 Episode initialization and reward reporting 16

B Bidding Algorithms **16**
B.1 MPC 16
B.2 GreedyMyopic 18
B.3 Shading 19

C Experimental details **20**
C.1 Scenario configuration 20
C.2 Evaluation protocol 22
C.3 Hyperparameters and training 22
C.4 Deep RL training details 24

D Additional experiments **27**
D.1 Bidding under varying auction frequency and recovering user fatigue 27
D.2 Computational cost and performance of bidding policies 28
D.3 Robustness of MPC to valuation errors and fewer rollouts 29
D.4 Evaluating bidding policies without delayed rewards 31
D.5 Evaluating bidding policies against the optimal policy under saturation and recency 31

A Environment implementation and configuration

DynamAuction provides seven predefined scenarios (Table 2), each of which can be customized using the parameters listed in Tables 5 and 6. Our main experiments use the `survival-second-price-exponential` scenario. Additional experiments consider immediate conversions and the saturation and recency scenarios (Appendix D).

Preset ID	Kernel	Carry-over context
`survival-second-price-exponential`	Survival	multiplicative fatigue
`survival-recovering-second-price-exponential-burst-arrivals`	Survival	recovering multiplicative fatigue
`survival-second-price-exponential-immediate`	Immediate survival	multiplicative fatigue
`saturation-second-price-lognormal`	Saturation	number of wins
`recency-second-price-lognormal`	Recency	time of the last win
`effective-frequency-second-price-lognormal`	Effective frequency	number of wins
`reminder-second-price-lognormal`	Reminder	time of the last win

Table 2: Built-in scenarios available in DYNAMAUCTION, with their reward kernels and carry-over contexts. Unless otherwise specified, all scenarios use second-price auctions, an exponentially distributed campaign duration with mean $\eta = 50$, and competing bids drawn from $\mathrm{LogNormal}(-1, 1)$. See the GitHub repository for the full scenario configurations

.

A.1 Gymnasium Interface and Wrappers

DYNAMAUCTION follows the Gymnasium API and can be used with Stable-Baselines3. For RL training, we apply three wrappers in order: remove the oracle expected reward from the observation, flatten the remaining observation fields into a vector, and clip bids to $[0, b_{\max}]$ (Table 3).

Class	Type	Effect
`MaskExpectedRewardWrapper`	Observation	Removes `expected_reward` from the observation dict. Prevents RL agents from accessing the oracle conversion value that FIPLA bids directly, ensuring a fair comparison.
`FlatAuctionObsWrapper`	Observation	Concatenates all remaining `Dict` observation fields into a single 1-D `float32` vector, compatible with standard MLP policies.
`BidScaleWrapper`	Action	Clips the action space from $[0, 10^{12}]$ to $[0, b_{\max}]$ (default $b_{\max} = 5.0$), restricting exploration to a practically relevant bid range.

Table 3: Observation and action wrappers provided by DYNAMAUCTION. The recommended stacking order for RL training is top to bottom.

A.2 Observation and Action Spaces

Table 4 describes the observation fields provided by the environment. The user features in Z_i (Section 3.4) consist of an observed user vector $u_{\mathrm{obs}} \in \mathbb{R}^{d_o}$, with $d_o \leq d$, while the reward kernel depends on the full vector $u \in \mathbb{R}^d$. In all our experiments, the user features are fully observed ($d_o = d$).

Table 4: Observation variables of the DYNAMAUCTION environment. Fields marked with † describe the previous auction and are excluded from policy inputs. The oracle value $V_0(Z_i)$ is used by FIPLA but masked from RL observations.

Quantity	Dim.	Description
θ_i	1	Current auction time
$\phi(S_u(\theta_i^-), \theta_i)$	1	Carry-over summary just before the auction (Section 3): the context $S_u(\theta_i^-)$ itself when it is fully observed, or a scalar computed from it.
i	1	Auction index within episode
u_{obs}	d_o	Observable user features
x_i	d	Display feature vector
$y_u(\theta_i)$	1	Exogenous process value
$\mathbb{E}[\text{reward} \mid \text{win}_i, x_i, u]$	1	Oracle expected reward
$\tau_1, \ldots, \tau_K$	K	Times of the delayed conversions matured since θ_{i-1}: the K most recent ones, zero-padded if fewer (K = `max_history`, 1 in all presets)
$R_{[i-1,i]}$	1	Cumulative reward generated by the delayed conversions
$B_i^\dagger$	1	Highest competing bid
$\mathbf{1}[\text{win}_i]^\dagger$	1	Win indicator
$p_i^\dagger$	1	Payment

A.3 Episode initialization and reward reporting

At the start of each episode, the environment samples the campaign horizon T and the auction times over $[0, T]$. Each step corresponds to an auction and advances the simulation to the next auction time. When an auction is won, the conversion times of the corresponding display are sampled. Delayed conversions are reported at the next auction, without identifying the display that generated them. Let C_i^{now} denote the number of immediate conversions from winning auction i, and $C_{[i-1,i]}^{\text{delayed}}$ the number of delayed conversions occurring in $(\theta_{i-1}, \theta_i]$, with $\theta_0 = 0$. The reward reported at step i is

$$\tilde{r}_i = vW_iC_i^{\text{now}} - p_i + vC_{[i-1,i]}^{\text{delayed}}, \tag{12}$$

where v is the value of a conversion, W_i indicates whether auction i is won, and $p_i = p(b_i, B_i)$ is the payment, which is zero if the auction is lost. Unlike the expected reward $r_j(h)$ associated with a single display in Equation (3), the step reward $\tilde{r}_i$ may include conversions from several earlier wins. The episode ends after the last auction before T, and conversions occurring after this auction are not credited, even if they occur before T.

B Bidding Algorithms

We provide pseudocode and implementation details for MPC, followed by the GreedyMyopic and Shading baselines.

B.1 MPC

At auction i, which takes place at time θ_i, the bidder receives the observation Z_i. The learned estimator $\widehat{V}_0(Z_i)$ predicts the conversion value of the current display, before payment. MPC uses this estimate to construct candidate bids, then compares them by simulating their consequences.

Candidate bids and future decisions. Let $\mathcal{A}_0$ be a finite set of nonnegative multipliers. For each $\alpha \in \mathcal{A}_0$, the candidate bid is $\max(0, \alpha\widehat{V}_0(Z_i))$. We evaluate this bid with M simulated trajectories, called rollouts. Each rollout processes the current auction and the following auctions up to time $\theta_i + H$, where the planning horizon H is measured in time units.

After the current auction, bids follow a fixed continuation policy. At a later simulated auction k, this policy bids $\max(0, \alpha_{\text{cont}}\widehat{V}_0(Z_k))$, where Z_k is the observation reached in that rollout and α_{cont} is a

fixed nonnegative multiplier. Thus, only the first multiplier α is optimized; subsequent bids depend on the simulated observations through $\widehat{V}_0$.

Simulated campaigns and shared random draws. Each rollout starts from a copy of the current simulator state, including past wins and pending conversions. The current observation is kept fixed, while future events are sampled anew. For the exponential campaign durations used here, rollout m draws a remaining duration $T'_m \sim \mathrm{Exp}(1/\eta)$, where η is the mean campaign length. It then samples the future auction times and exogenous events up to $\theta_i + T'_m$. We write $\theta_k^{(m)}$ for the auction times in this sampled campaign, with $\theta_i^{(m)} = \theta_i$.

The same sampled campaign is used for every candidate α. Competing bids and future display features are drawn from separate random streams, and conversion draws are indexed by auction. This keeps the random inputs at auction k aligned across candidates, even when earlier auction outcomes differ. The conversion outcomes can still differ, because their probabilities depend on the history of wins. These common random numbers are used to reduce the variance of differences between candidate returns.

Scoring a rollout. For each candidate α, rollout m simulates the current auction and subsequent auctions up to the planning horizon H. Its return is the net reward of the displays won during this window:

$$R^{(m)}(\alpha) = \sum_{k=i}^{k_{\mathrm{end}}^{(m)}} \mathcal{W}\big(b_k^{(m)}, B_k^{(m)}\big)\Big(C_k^{(m)} - p\big(b_k^{(m)}, B_k^{(m)}\big)\Big), \qquad b_k^{(m)} = \alpha_k^{(m)}\, \widehat{V}_0\big(Z_k^{(m)}\big),$$

where $\alpha_i^{(m)} = \alpha$ and $\alpha_k^{(m)} = \alpha_{\mathrm{cont}}$ for $k > i$. Here, $k_{\mathrm{end}}^{(m)}$ is the last auction within the planning horizon, and $C_k^{(m)}$ is the value of the conversions generated by the display won at auction k that occur before the last auction of the simulated campaign, at time $t^{(N_m)}$.

Importantly, the planning horizon limits which auctions are simulated, not which conversions are counted. A display won within the planning horizon may generate conversions after H, and these are included in its return as long as they occur before $t^{(N_m)}$, consistently with the environment's reward reporting. Conversions from displays won before auction i are omitted because they do not depend on the candidate α.

The estimated return of each candidate is then

$$\widehat{Q}(\alpha) = \frac{1}{M} \sum_{m=1}^{M} R^{(m)}(\alpha).$$

MPC selects the candidate with the largest mean score, breaking ties in favor of the smallest multiplier. It submits this bid and repeats the procedure at the next real auction. Algorithm 1 gives the steps.

Algorithm 1 MPC: selecting the bid at auction i

Require: Current simulator state, observation Z_i, auction time θ_i
Require: Estimator $\widehat{V}_0$, multipliers $\mathcal{A}_0$, continuation multiplier α_{cont}, horizon H, number of rollouts M

$v_i \leftarrow \widehat{V}_0(Z_i)$
for $m = 1, \dots, M$ **do**
    Draw a remaining duration $T'_m \sim \text{Exp}(1/\eta)$
    Sample future auctions and exogenous events up to $\theta_i + T'_m$
    $L_m \leftarrow$ last auction time in the sampled campaign, including θ_i
    $K_m \leftarrow$ last auction index with $\theta_k^{(m)} \le \theta_i + H$
    Store the shared random streams for competing bids, display features and conversions
end for
for each $\alpha \in \mathcal{A}_0$ **do**
    $\widehat{Q}(\alpha) \leftarrow 0$
    for $m = 1, \dots, M$ **do**
        Copy the current simulator state, keeping past wins and pending conversions
        Load rollout m's auction schedule and exogenous events; reset its random streams
        $R \leftarrow 0$
        for $k = i, \dots, K_m$ **do**
            Advance the copy to auction k and read its observation $Z_k^{(m)}$
            if $k = i$ **then**
                $b \leftarrow \max(0, \alpha v_i)$
            else
                $b \leftarrow \max(0, \alpha_{\text{cont}} \widehat{V}_0(Z_k^{(m)}))$
            end if
            Determine the outcome and payment p using b and the competing bid of rollout m at auction k
            if the auction is won **then**
                Sample and schedule the display's conversions using its pre-win context and the draws for (m, k)
                $C \leftarrow$ total value of these conversions occurring no later than L_m
                $R \leftarrow R + C - p$
            end if
            Update the simulator copy with the auction outcome
        end for
        $\widehat{Q}(\alpha) \leftarrow \widehat{Q}(\alpha) + R/M$
    end for
end for
$\alpha^* \leftarrow \min \arg\max_{\alpha \in \mathcal{A}_0} \widehat{Q}(\alpha)$
return $\max(0, \alpha^* v_i)$

Special cases. With $\mathcal{A}_0 = \{1\}$, MPC returns the bid $\max(0, \widehat{V}_0(Z_i))$; using the oracle V_0 gives the FIPLA bid. With $H = 0$, each rollout evaluates only the current auction, including its delayed conversions credited before L_m. In a second-price auction, $\alpha = 1$ is optimal in expectation if the valuation is exact for these credited conversions and $1 \in \mathcal{A}_0$. A learned valuation and a finite number of rollouts can lead to a different choice.

Computational cost. Each decision evaluates $|\mathcal{A}_0|M$ rollouts. The number of auctions processed in each rollout depends on H, the arrival process and the sampled campaign duration. The rollouts use the simulator's dynamics directly; $\widehat{V}_0$ is the learned component used to set bids.

B.2 GreedyMyopic

GreedyMyopic estimates the conversion value of winning the current auction, assuming that no further auctions will be won. Let π_0 denote the continuation policy that bids zero at every subsequent auction.

At auction i, we run M pairs of simulations from the current simulator state. In each pair, one simulation forces a win at auction i and the other forces a loss. Both then follow π_0 until the last auction of the simulated campaign. They share the same remaining duration $T'_m \sim \mathrm{Exp}(1/\eta)$, future auction times and exogenous events, and start with the same noise seed ω_m.

Let $R^{(m)}_{\mathrm{win}}$ and $R^{(m)}_{\mathrm{loss}}$ denote the total conversion value credited in these two simulations. Payments are excluded, since the quantity being estimated is the display's value before payment. As in the environment, conversions occurring after the last auction are not credited. The bid is

$$b_i = \hat{\Delta} = \max\left(0,\ \frac{1}{M}\sum_{m=1}^{M}\left(R^{(m)}_{\mathrm{win}} - R^{(m)}_{\mathrm{loss}}\right)\right).$$

The paired simulations use common random numbers to cancel the contributions of previously won displays, as detailed in Remark 1.

Algorithm 2 GreedyMyopic: selecting the bid at auction i

Require: Current simulator state at auction i (observation Z_i), zero-bid continuation policy π_0, number of pairs M
Ensure: Bid $b_i \geq 0$

$S_{\mathrm{win}} \leftarrow 0$; $S_{\mathrm{loss}} \leftarrow 0$
for $m = 1, \ldots, M$ **do**
  Draw a scenario seed and a noise seed ω_m
  Use the scenario seed to sample $T'_m \sim \mathrm{Exp}(1/\eta)$, future auction times and exogenous events
  Forced win:
  Copy the current simulator state, keeping past wins and pending conversions
  Load the sampled campaign and initialize the noise generator with ω_m
  Force auction i to be won
  Follow π_0 through the last auction of the sampled campaign
  $R^{(m)}_{\mathrm{win}} \leftarrow$ total conversion value credited in this simulation
  Forced loss:
  Make a fresh copy of the same current simulator state
  Load the same sampled campaign and reset the noise generator to ω_m
  Force auction i to be lost
  Follow π_0 through the last auction of the sampled campaign
  $R^{(m)}_{\mathrm{loss}} \leftarrow$ total conversion value credited in this simulation
  $S_{\mathrm{win}} \leftarrow S_{\mathrm{win}} + R^{(m)}_{\mathrm{win}}$
  $S_{\mathrm{loss}} \leftarrow S_{\mathrm{loss}} + R^{(m)}_{\mathrm{loss}}$
end for
return $\max\left(0,\ \dfrac{S_{\mathrm{win}} - S_{\mathrm{loss}}}{M}\right)$

Remark 1 (Common Random Numbers)**.** *The scenario seed fixes the remaining campaign duration, auction times and exogenous events. The noise seed ω_m initializes the single generator used for competing bids, display features and conversions. The forced-win simulation draws conversions for the current display, which shifts the subsequent draws of this generator. These later draws do not affect conversion rewards because π_0 wins no further auctions. Conversions already pending at auction i are copied unchanged into both simulations and are credited at the same auction times. Their contributions therefore cancel in $R^{(m)}_{\mathrm{win}} - R^{(m)}_{\mathrm{loss}}$, leaving the conversion value credited to the current display. This value remains random, and $\hat{\Delta}$ averages it over the M pairs.*

B.3 Shading

Shading uses the same value estimate $\hat{\Delta}$ as GreedyMyopic (Algorithm 2), but multiplies it by a fixed factor $\alpha \in (0, 1]$. This reduces the bid to account for the effect of winning the current auction on the value of future opportunities. When $\alpha = 1$, Shading is equivalent to GreedyMyopic.

Algorithm 3 Shading(α) — bid at auction i

Require: current state Z_i, shading factor $\alpha \in (0, 1]$, GreedyMyopic estimator (Algorithm 2)
Ensure: bid $b_i \geq 0$
1: $\hat{\Delta} \leftarrow \text{GreedyMyopic}(Z_i)$
2: **return** $\max(0,\ \alpha \cdot \hat{\Delta})$

C Experimental details

C.1 Scenario configuration

Tables 5 and 6 list the configurable components of DYNAMAUCTION and their preset values. The main experiments use this preset with $\eta = 50$.

Table 5: Configurable components of a DYNAMAUCTION scenario: auction, horizon, competition, arrival, user features, and display features. A dash (—) indicates inactive components.
(a) Auction, horizon, competition, arrival

Component	Parameter	Options / Type	Preset
Auction	mechanism	`second-price`, `first-price`	second-price
Horizon	type	`exponential`, `fixed`	exponential
	η (`exponential`)	$\mathbb{R}_+$, mean campaign length	50
	T (`fixed`)	$\mathbb{R}_+$, campaign length	—
Competition	distribution	`lognormal`, `uniform`, `exponential`, `constant`	lognormal
	μ_B, σ_B (`lognormal`)	$\mathbb{R}$, $\mathbb{R}_+$	-1.0, 1.0
	low, high (`uniform`)	$\mathbb{R}_+$	—
	scale (`exponential`)	$\mathbb{R}_+$	—
	value (`constant`)	$\mathbb{R}_+$, the same competing bid at every auction	—
Arrival	process	`cstpoisson`, `pwpoisson`, `decaypoisson`	`cstpoisson`
	rate μ (`cstpoisson`)	$\mathbb{R}_+$	1.0
	breakpoints, intensities (`pwpoisson`)	piecewise-constant rate: lists in $\mathbb{R}_+$	—
	rate, period, factor (`decaypoisson`)	rate $\times$ factor$^{\lfloor t/\text{period} \rfloor}$, in $\mathbb{R}_+$	—

(b) User and display features

Component	Parameter	Options / Type	Preset
User features	sampler type	`uniform`, `normal`, `fixed`	normal
	full dim d	$\mathbb{N}$	3
	observable dim $d_o \leq d$	$\mathbb{N}$	3
	mean μ_u	$\mathbb{R}$	0.0
	std σ_u	$\mathbb{R}_+$	1.0
	feature vector u	$\mathbb{R}^d$	—
Display features	sampler type	`iid-uniform`, `catalogue`, `fixed`, `normal`	normal
	IIDUniformSampler	$x_i \sim \mathcal{U}([0,1]^d)$	—
	CatalogueSampler	$x_i \sim \mathcal{C}$	—
	FixedSampler	$x_i = x_0$	—
	NormalSampler	$x_i \sim \mathcal{N}(\mu, \sigma^2 I)$	$\mu = 0$, $\sigma = 1$

Table 6: Configurable components of a DYNAMAUCTION scenario: value model, context, exogenous process, and conversion generation.

(c) Value model, context, and exogenous process

Component	Parameter	Options / Type	Preset
Value kernel	type	`survival`, `immediate-survival`, `saturation`, `recency`, `competitor-extinction`, `effective-frequency`, `reminder`, `mixed`	survival
	$\mathbf{w}_u, \mathbf{w}_x$ (`survival`, `immediate-survival`; optional relevance for `effective-frequency`, `reminder`)	$\mathbb{R}^d$	$\mathbf{1}, \mathbf{1}$
	delay rate δ (all but `mixed`)	$\mathbb{R}_+$	1.0
	base rate, saturation scale (`saturation`)	$\mathbb{R}_+$	—
	recency rate (`recency`)	$\mathbb{R}_+$	—
	base rate (`competitor-extinction`)	$\mathbb{R}_+$	—
	base rate, wear-in midpoint and slope, saturation scale (`effective-frequency`)	$\mathbb{R}_+$	—
	p_{cold}, p_{peak}, peak time (`reminder`)	$[0, 1]$, $[0, 1]$, $\mathbb{R}_+$	—
	base kernel, κ_{base} (`mixed`)	kernel type, $\mathbb{R}_+$	—
Carry-over S_u and decay map γ	type	`multiplicative`, `recovering-multiplicative`, `additive`, `last-won-time`, `time-since-last-win`, `constant`	multiplicative
	decay c (`multiplicative`, `recovering-multiplicative`)	$(0, 1)$	0.9
	recovery rate β (`multiplicative`, `recovering-multiplicative`)	$\mathbb{R}_+$	1.0
	initial value (`multiplicative`, `additive`, `last-won-time`, `time-since-last-win`)	$\mathbb{R}$, context before the first win	1.0
	value (`constant`)	$\mathbb{R}$	—
Exogenous process	type	`trivial` ($y(t) = 0$), `competitor-click`	trivial
	click rate (`competitor-click`)	$\mathbb{R}_+$, Poisson rate of competitor clicks; $y(t)$ is the time of the last click	—

(d) Conversion generation and delay modeling

Component	Parameter	Options / Type	Preset
Conversion probe	type	`poisson`, `bernoulli`, `null`	bernoulli
Poisson probe	sampling method	Ogata thinning process	—
	event generation	point process over horizon	—
Bernoulli probe	conversion gate	Bernoulli + exponential delay	✓
	max conversions	0 or 1 per display	—
Null probe	rule	no delayed conversion (immediate kernels)	—
Delay sampler	type	`single`, `multi`	single
Single sampler	rule	at most one conversion possible per display	✓
Multi sampler	rule	multiple conversions allowed	—
Display history	`max_history`	$\mathbb{N}$	1

C.2 Evaluation protocol

All policies are evaluated on the same 1 000 test campaigns, generated with environment seeds 0–999. This allows us to compare policies on identical campaigns, rather than on independently sampled ones. For stochastic policies, campaign s uses the policy seed $s + 1000$.

Training and model selection use different campaigns. The MPC valuation head is trained on 1 000 FIPLA campaigns (seeds 10 000–10 999), while the RL agents generate their training campaigns from their own random streams. Intermediate evaluations during RL training use seeds $\geq 10\,000$. RL hyperparameters and checkpoints are selected using 200 validation campaigns (seeds 20 000–20 199). Each selected RL configuration is then trained with three different random seeds.

We report mean rewards with 95% confidence intervals, written as the mean $\pm 1.96\,\hat{\sigma}/\sqrt{n}$, where $\hat{\sigma}$ is the sample standard deviation over the n test campaigns. For comparisons between policies A and B, we compute the reward difference $R_A(s) - R_B(s)$ on each campaign s and use the standard deviation of these paired differences to construct the confidence interval. For RL policies, $R(s)$ is the average reward of the three trained models on campaign s. Since both policies face the same campaigns, pairing reduces the variability due to differences in campaign length and user characteristics, yielding narrower confidence intervals for their performance differences.

C.3 Hyperparameters and training

We report the configurations used in the experiments and how their parameters were selected.

MPC. MPC evaluates candidate bids using the multiplier grid $\mathcal{A}_0 = \{0.1, 0.2, \ldots, 1.0\}$, with ties resolved in favor of the smallest multiplier. We consider planning horizons $H \in \{10, 25, 35\}$, $M \in \{60, 80\}$ rollouts per candidate, and continuation multipliers $\alpha_{\text{cont}} \in \{1, 0.4\}$, where 0.4 is the multiplier selected for Shading. Unless otherwise specified, $\alpha_{\text{cont}} = 1$ in all tables and figures.

As described in Algorithm 1, candidate bids are compared using common random numbers for competing bids, display features, and conversions. Each rollout is scored by the net reward of the displays won within the planning horizon, including their conversions occurring before the last auction of the simulated campaign.

MPC uses a learned valuation head $\widehat{V}_0$ to estimate the oracle value V_0 from observations that exclude V_0 itself. We train this estimator on 1 000 FIPLA campaigns, corresponding to 49 358 auctions at $\eta = 50$, with 15% of the data held out for validation. Training uses Adam with a learning rate of 10^{-3}, a batch size of 256, and 500 epochs.

We compare MLPs with two hidden layers of 32, 64, or 128 units, using ReLU or Leaky ReLU activations (Figure 3 and Table 7). Although the larger networks achieve lower training MSE (0.00097 and 0.00068, compared with 0.00147 for 32-32 ReLU), their test errors are higher (0.00208 and 0.00209, compared with 0.00184). Leaky ReLU performs similarly to ReLU (test MSE 0.00181 for 32-32 Leaky ReLU). We therefore use the 32-32 ReLU network, which has the lowest test error among the ReLU architectures and requires only 1 505 parameters, compared with 5 057 and 18 305 for the larger networks. Its smaller size also reduces the cost of the repeated predictions made during MPC rollouts.

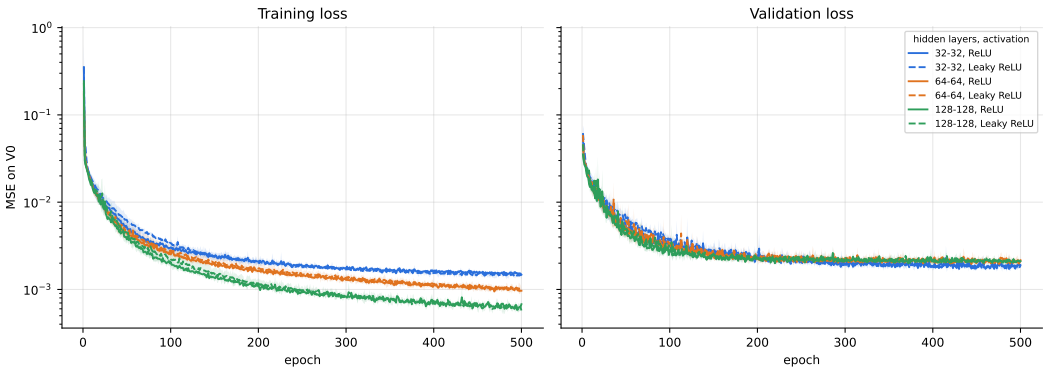


Figure 3: Training and validation mean squared error (MSE) of MLPs used to estimate the oracle value V_0. We compare three network sizes (32-32, 64-64, and 128-128 hidden units) with ReLU and Leaky ReLU activations. All models are trained and evaluated on the same data split from the main scenario. Curves show the mean over three random initializations, with shaded areas indicating the minimum and maximum values. MPC uses the 32-32 ReLU network.

Table 7: MLP valuation heads of different sizes and activations (mean over 3 initialization seeds). Test: 200 new FIPLA campaigns.

hidden layers	activation	parameters	train MSE	validation MSE	test MSE	test corr.	minutes
32-32	relu	1505	0.00147	0.00190	0.00184	0.988	5.7
32-32	leaky relu	1505	0.00145	0.00186	0.00181	0.988	5.7
64-64	relu	5057	0.00097	0.00216	0.00208	0.986	6.0
64-64	leaky relu	5057	0.00101	0.00216	0.00227	0.986	6.0
128-128	relu	18305	0.00068	0.00211	0.00209	0.987	6.7
128-128	leaky relu	18305	0.00057	0.00207	0.00208	0.987	6.5

GreedyMyopic and Shading. Both policies estimate the value of the current display as the mean difference between M Monte Carlo rollouts that force a win and M that force a loss, continued with the bid 0; each pair of rollouts shares its random numbers, so that environment noise cancels in the difference. The shading multiplier is selected by a grid search over $\alpha \in \{0.1, 0.2, \ldots, 1.0\}$ with $M = 20$ rollouts, on the 1 000 test campaigns: the best value is $\alpha^* = 0.4$ (Figure 4), and GreedyMyopic is Shading with $\alpha = 1$ and $M = 20$. Unlike the RL hyperparameters, which are selected on validation campaigns, the Shading multiplier (and hence $\alpha_{\text{cont}} = 0.4$ in MPC) and the constant bid are selected on the test campaigns, which slightly favors these policies.

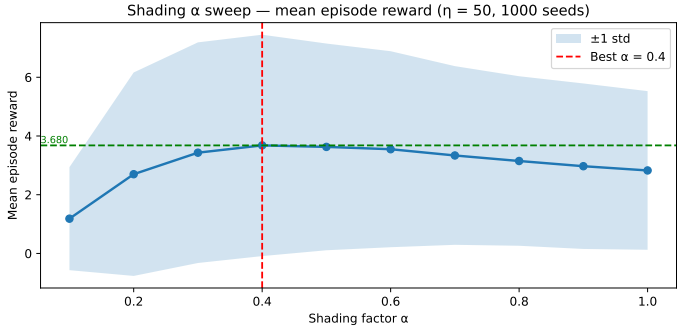
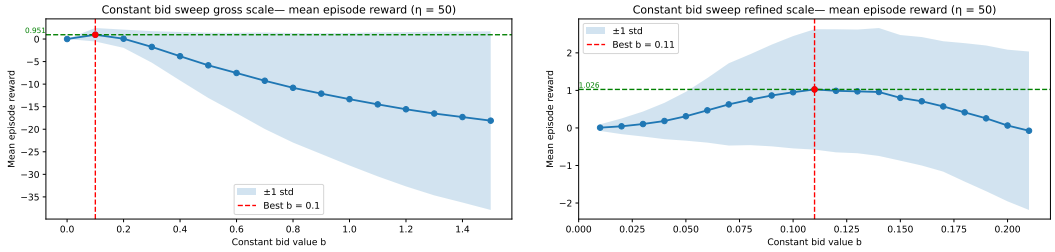


Figure 4: Average episode reward of Shading(α) with $M = 20$ rollouts as a function of the shading multiplier α, evaluated over the 1 000 test campaigns at $\eta = 50$. The shaded band shows ± 1 standard deviation across episodes.

Constant bidder. The constant bid is selected by a two-stage grid search on the 1 000 test campaigns, first on a coarse grid of step 0.1 over $[0, 1.5]$, then on a grid of step 0.01 within ± 0.1 of the best coarse value: $b^* = 0.11$ (Figure 5).



(a) Coarse grid, $b^* = 0.1$, reward = 0.951

(b) Refined grid, $b^* = 0.11$, reward = 1.026

Figure 5: Average episode reward as a function of the constant bid b at $\eta = 50$, using two grid resolutions: coarse (b on a 0.1 grid) and refined (0.01 grid within ± 0.1 of the best coarse value). Results are evaluated over the 1 000 test campaigns. The shaded band shows ± 1 standard deviation across episodes.

C.4 Deep RL training details

Observations and actions. PPO, SAC, and TD3 receive flattened observations from which the oracle value used by FIPLA has been removed. This value is unavailable during both training and evaluation. Observations and rewards are normalized using `VecNormalize`, and bids are clipped to $[0, 5]$. We use the default Stable-Baselines3 network architectures: two hidden layers of 64 units with $\tanh$ activations for PPO, and two hidden layers of 256 units with ReLU activations for SAC and TD3.

Hyperparameter search. We select the hyperparameters of each algorithm through a grid search at $\eta = 50$, using the Inria Margaret cluster. The search covers 144 configurations for PPO, 48 for SAC, and 96 for TD3. The tested values and fixed parameters are listed in Table 8.

Each configuration is trained for 5×10^6 timesteps with seed 42 and four parallel environments. During training, we evaluate the policy on 20 campaigns every 10^4 timesteps and save the checkpoint with the highest evaluation reward. At the end of training, we compare this checkpoint with the final checkpoint on a separate set of 200 validation campaigns, using the normalization statistics saved with each checkpoint. For each algorithm, we select the configuration and checkpoint with the highest mean validation reward.

Final training. Once the hyperparameters have been selected, we train the chosen configuration again for 10^7 timesteps with three different seeds (42, 43, and 44). For each training, we compare the best intermediate checkpoint and the final checkpoint on the 200 validation campaigns, then evaluate the better one on the 1 000 test campaigns. The results reported for each RL algorithm are averaged over these three independently trained models.

Table 8 reports the selected hyperparameters, their validation rewards during the search, and the test reward obtained by each of the three final trainings.

Table 8: Hyperparameter search for PPO, SAC, and TD3 at $\eta = 50$. The second column lists the values tested for each parameter or its fixed value. Bold entries indicate the hyperparameters selected using the 200 validation campaigns. The last rows report the validation reward of the selected configuration during the search and the test rewards of its three final trainings.

Parameter	Search grid / Fixed	PPO best	SAC best	TD3 best
Shared				
Total timesteps	search 5×10^6; final 10^7 (3 trainings)		—	
Parallel environments	4		—	
Normalization	VecNormalize (observations, rewards)		—	
Eval. during training	20 campaigns every 10^4 steps		—	
Selection	200 validation campaigns, best or final checkpoint		—	
PPO				
Learning rate	$\{5\times10^{-5}, 10^{-4}, 2\times10^{-4}, 3\times10^{-4}\}$	$\mathbf{5 \times 10^{-5}}$	—	—
Rollout length (steps)	$\{256, 512\}$	**256**	—	—
Optimization epochs	$\{2, 5, 10\}$	**10**	—	—
Entropy coefficient	$\{0, 0.01, 0.02\}$	**0**	—	—
Discount factor	$\{0.95, 0.99\}$	**0.99**	—	—
Mini-batch size	128 (fixed)	—	—	—
GAE parameter	0.90 (fixed)	—	—	—
Clip range ϵ	0.20 (fixed)	—	—	—
SAC				
Learning rate	$\{5\times10^{-5}, 10^{-4}, 3\times10^{-4}, 10^{-3}\}$	—	$\mathbf{5 \times 10^{-5}}$	—
Batch size	$\{256, 512\}$	—	**512**	—
Learning starts (warmup)	$\{500, 1000, 1500\}$	—	**500**	—
Discount factor	$\{0.95, 0.99\}$	—	**0.99**	—
Entropy coefficient	auto-tuned	—	—	—
TD3				
Learning rate	$\{5\times10^{-5}, 10^{-4}, 3\times10^{-4}, 10^{-3}\}$	—	—	$\mathbf{10^{-3}}$
Batch size	$\{256, 512\}$	—	—	**512**
Exploration noise σ	$\{0.1, 0.2\}$	—	—	**0.2**
Learning starts (warmup)	$\{500, 1000, 1500\}$	—	—	**500**
Discount factor	$\{0.95, 0.99\}$	—	—	**0.99**
Policy update delay	2 (fixed)	—	—	—
Target policy noise	0.2 (fixed)	—	—	—
Total grid configurations		144	48	96
Validation reward of the selected configuration		4.174	0.863	3.769
Test reward of the 3 trainings		3.52 / 3.53 / 1.84	0.95 / 0.74 / 0.43	2.89 / 3.13 / 0.00

Results of the hyperparameter search. Figure 6 shows the validation reward obtained for each tested configuration. Most TD3 configurations converge to policies that never bid (82 out of 96), as do 5 of the 48 SAC configurations. PPO is less affected by this failure mode, although its final performance still depends on the training seed, as discussed below.

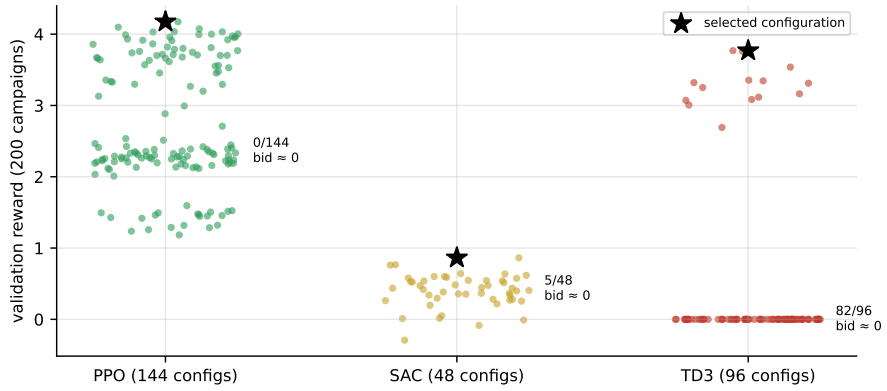


Figure 6: Mean reward on 200 validation campaigns for each hyperparameter configuration tested at $\eta = 50$. For each configuration, the better of the best intermediate and final checkpoints is shown. Stars indicate the configurations selected for the final trainings. Among the tested configurations, 82 of 96 for TD3 and 5 of 48 for SAC converge to policies that never bid (reward approximately zero).

Training behavior. Figure 7 shows the learning curves of the three final trainings for each algorithm. The runs reveal substantial differences across algorithms and training seeds.

TD3 is particularly unstable: one of its three final trainings converges to a policy that never bids, as observed in most configurations during the search. Once its deterministic bid falls to zero, the agent stops winning auctions and does not recover in these runs.

PPO also shows substantial variability across seeds, with test rewards of 3.52, 3.53, and 1.84 despite using the same hyperparameters. SAC, by contrast, reaches its final performance level within approximately 10^6 steps, but learns a policy that wins many cheap auctions and keeps users fatigued.

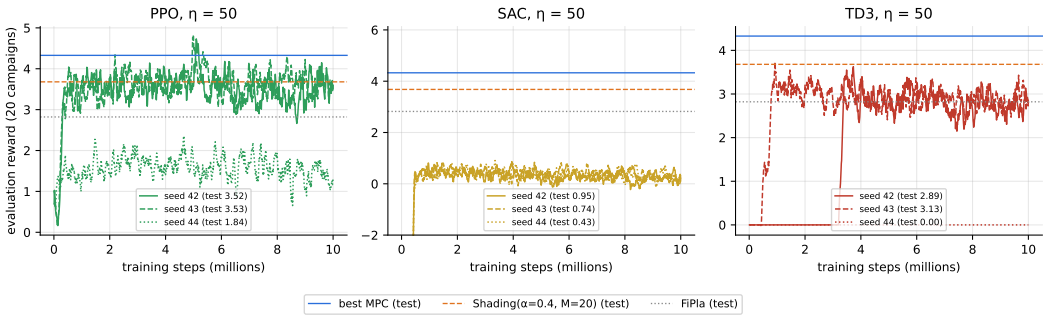


Figure 7: Evaluation reward during the three final trainings of PPO, SAC, and TD3 at $\eta = 50$. Policies are evaluated on 20 campaigns every 10^4 steps; curves show a rolling mean over 10 evaluations. The legend reports the test reward of each training on 1 000 campaigns. Horizontal lines indicate the test rewards of the best MPC configuration, Shading(0.4, $M = 20$), and FiPla. The earliest SAC evaluations, with rewards as low as -50, are omitted from the vertical scale.

Longer PPO training. To check whether PPO benefits from a larger training budget, we extend the selected configuration to 5×10^7 timesteps, using the same three training seeds. Figure 8 shows that test performance decreases over the extended runs. For example, seed 42 achieves a test reward of 3.59 after 5×10^6 steps, 3.31 after 10^7 steps, and 2.61 after 5×10^7 steps. Increasing the training budget therefore does not improve PPO's performance in this experiment.

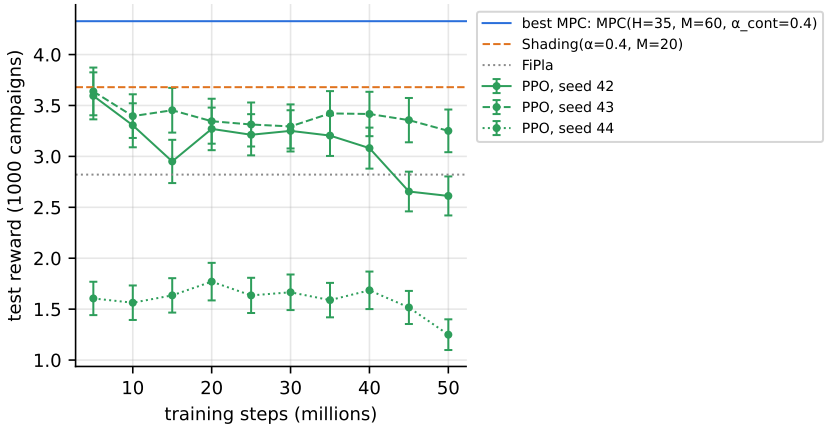


Figure 8: Test reward of the selected PPO configuration trained for 5×10^7 steps at $\eta = 50$, using three training seeds. Each point corresponds to a checkpoint saved every 5×10^6 steps and evaluated on the same 1 000 test campaigns. Error bars show 95% confidence intervals.

D Additional experiments

D.1 Bidding under varying auction frequency and recovering user fatigue

We consider a variant of the main scenario to test whether MPC adjusts its bids when the cost of winning changes over the course of a campaign. Two components are modified :

- *Recovering fatigue.* Each past win at time t_i reduces the value of future displays through a factor that gradually returns to 1:

$$S_u(t^-) = \prod_{i:\, t_i < t} \big(1 - ce^{-\beta(t-t_i)}\big), \qquad c = 0.9, \quad \beta = 0.3.$$

  This corresponds to Equation (1) with the monoid $([0,1], \times, 1)$ and decay map $\gamma(x_i, t_i, t) = 1 - ce^{-\beta(t-t_i)}$. Each factor starts at 0.1 immediately after a win and approaches 1 independently of subsequent wins. After five time units, a win still reduces conversion propensity by about 20%. The reward kernel uses $S_u(t^-)$ instead of ϕ, giving a conversion propensity $\mathrm{clip}\big(S_u(t^-)(\mathbf{1}^\top u)(\mathbf{1}^\top x), 0, 1\big)$. The bidder observes $S_u(t^-)$, but not the past win times that determine its future evolution.
- *Higher auction frequency at the start of the campaign.* Auctions arrive according to a non-homogeneous Poisson process with rate 10 on $[0, 10)$ and rate 1 thereafter, instead of rate 1 throughout the campaign. The campaign duration remains exponential with mean $\eta = 50$. This produces 131 auctions per campaign on average, including 92 during the initial high-frequency period.

During the first ten time units, auctions arrive roughly 0.1 time units apart, so fatigue from successive wins accumulates before it can dissipate. Later, auctions are less frequent and users have more time to recover. Winning an auction can therefore reduce the value of many more future opportunities early in the campaign than later on, making a fixed shading multiplier potentially suboptimal.

Experimental protocol. All policies are evaluated on the same 200 test campaigns (environment seeds 0–199, policy seed $s+1000$ for campaign s). Comparisons are paired, as in the main benchmark, but use one fifth as many campaigns, giving confidence intervals approximately 2.2 times wider.

Shading uses $M = 20$ rollouts, with its multiplier selected from $\{0.1, \dots, 1\}$ on these test campaigns ($\alpha = 0.6$). GreedyMyopic corresponds to $\alpha = 1$, while the two-stage search for the constant bidder gives $b = 0.09$.

MPC uses $M = 60$ rollouts per candidate, with $\alpha \in \{0.1, \dots, 1\}$, and a 32-32 ReLU valuation head trained on 1 000 FIPLA campaigns of this scenario, following the main benchmark procedure. We compare three continuation policies:

- *Fixed:* $\alpha_{\mathrm{cont}} = 0.6$, the selected Shading multiplier, with $H \in \{5, 15, 25\}$.
- *Shared:* the candidate multiplier α is also used for subsequent bids in each rollout, with $H \in \{5, 10, 15, 25\}$.
- *Two-phase:* $\alpha_{\mathrm{cont}} = 0.3$ before $t = 10$ and 0.7 thereafter, with $H \in \{5, 15, 25\}$.

The two-phase coefficients are selected by searching pairs (a_1, a_2) on a grid with step 0.1, using an oracle policy that bids $a_1 V_0$ before $t = 10$ and $a_2 V_0$ afterwards. Selection uses 200 validation campaigns (seeds 20 000–20 199).

For PPO, we conduct a smaller hyperparameter search centered on the configuration selected in the main benchmark. The initial grid combines learning rates $\{5 \times 10^{-5}, 10^{-4}, 3 \times 10^{-4}\}$, rollout lengths $\{256, 512\}$ and optimization epochs $\{5, 10\}$ (12 configurations). Since its best configuration uses 512 steps and 10 epochs, at the edge of this grid, we extend the search to learning rates $\{10^{-4}, 2 \times 10^{-4}, 3 \times 10^{-4}\}$, rollout lengths $\{512, 1024, 2048\}$ and epochs $\{5, 10, 20\}$. This adds 23 new configurations, for 35 in total. The entropy coefficient remains 0 and $\gamma = 0.99$; all other settings follow Table 8.

Each configuration is trained for 3×10^6 steps and evaluated on 200 validation campaigns using its best intermediate and final checkpoints. The selected configuration (learning rate 10^{-4}, 1 024 steps per update, 5 epochs, final checkpoint) is retrained with three seeds for 5×10^6 steps, and

each training is evaluated on the 200 test campaigns. Both training budgets are shorter than in the main benchmark (5×10^6 and 10^7 steps, respectively). SAC and TD3 are not evaluated here, as both performed below PPO in the main benchmark.

Table 9: Burst scenario: mean episode reward ($\pm$ 95% CI) over the first 200 test campaigns, and paired differences (RL: mean of the 3 trainings; the test reward of each training in parentheses). MPC continuation (multiplier of the bids after the first one in each rollout): $\alpha_{\text{cont}} = 0.6$ (the best Shading multiplier), shared α (each rollout bids its candidate α at every auction), or two-phase ($\alpha_{\text{cont}} = 0.3$ for $t < 10$, then 0.7).

Policy	reward	Δ vs Shading($\alpha = 0.6$)	Δ vs FIPLA
MPC($H = 5$, $M = 60$, $\alpha_{\text{cont}} = 0.6$)	6.531 ± 0.982	$+0.44 \pm 0.26$	$+1.14 \pm 0.26$
MPC($H = 15$, $M = 60$, $\alpha_{\text{cont}} = 0.6$)	6.471 ± 0.987	$+0.38 \pm 0.27$	$+1.08 \pm 0.30$
MPC($H = 25$, $M = 60$, $\alpha_{\text{cont}} = 0.6$)	6.453 ± 0.973	$+0.36 \pm 0.27$	$+1.06 \pm 0.30$
MPC($H = 5$, $M = 60$, shared α)	6.445 ± 0.974	$+0.35 \pm 0.28$	$+1.05 \pm 0.29$
MPC($H = 10$, $M = 60$, shared α)	6.444 ± 0.970	$+0.35 \pm 0.27$	$+1.05 \pm 0.30$
MPC($H = 15$, $M = 60$, shared α)	6.354 ± 0.960	$+0.26 \pm 0.26$	$+0.96 \pm 0.29$
MPC($H = 25$, $M = 60$, shared α)	6.165 ± 0.943	$+0.07 \pm 0.24$	$+0.77 \pm 0.28$
MPC($H = 5$, $M = 60$, two-phase α_{cont})	6.456 ± 0.953	$+0.36 \pm 0.28$	$+1.06 \pm 0.31$
MPC($H = 15$, $M = 60$, two-phase α_{cont})	6.494 ± 0.951	$+0.40 \pm 0.28$	$+1.10 \pm 0.31$
MPC($H = 25$, $M = 60$, two-phase α_{cont})	6.473 ± 0.953	$+0.38 \pm 0.29$	$+1.08 \pm 0.31$
PPO (5.80 / 3.31 / 5.72)	4.947 ± 0.709	-1.14 ± 0.39	-0.44 ± 0.43
Shading($\alpha = 0.6$, $M = 20$)	6.091 ± 0.944	—	$+0.70 \pm 0.28$
FIPLA	5.392 ± 0.956	-0.70 ± 0.28	—
GreedyMyopic	5.173 ± 0.901	-0.92 ± 0.26	-0.22 ± 0.25
Constant($b = 0.09$)	1.248 ± 0.242	-4.84 ± 0.79	-4.14 ± 0.81

Results. Shading improves on FIPLA by $+0.70 \pm 0.28$ (+13%). MPC performs better than Shading across the three continuation policies. Its best variant ($H = 5$, $M = 60$, $\alpha_{\text{cont}} = 0.6$) gains $+0.44 \pm 0.26$ (+7%) over Shading and $+1.14 \pm 0.26$ (+21%) over FIPLA. The 95% confidence interval for the gain over Shading excludes zero for 8 of the 10 MPC variants.

MPC bids more selectively during the high-frequency period, winning 5.7% of auctions, compared with 6.6% for Shading and 7.7% for FIPLA. After $t = 10$, its win rate rises to 17.9%, close to Shading's 17.6%. With the shared continuation, MPC selects an average multiplier of 0.45 before $t = 10$ and 0.74 afterwards, close to the oracle two-phase coefficients $(0.3, 0.7)$.

However, allowing the continuation policy to vary with time brings no clear improvement over the fixed multiplier $\alpha_{\text{cont}} = 0.6$. Most variants perform similarly, and the shared continuation at $H = 25$ is the only one significantly below the best fixed-continuation variant (-0.37 ± 0.23). Replanning at every auction appears sufficient to adapt the current bid to the campaign phase, without requiring a time-dependent continuation policy or a longer planning horizon.

PPO remains below Shading on average (-1.14 ± 0.39) and below the best MPC (-1.58 ± 0.41). Two of its three trainings outperform FIPLA ($+0.41 \pm 0.33$ and $+0.33 \pm 0.38$) but remain below Shading (-0.29 ± 0.29 and -0.37 ± 0.36). The third learns to bid too little, winning only 5.2% of auctions, compared with 9.3–12.2% for the other policies except Constant.

The selected PPO run reaches a validation reward of 6.62 after 3×10^6 steps, but its three retrainings achieve only 6.10, 2.46 and 6.29 on the same validation campaigns after 5×10^6 steps. The high reward of the selected run thus appears optimistic, reflecting the selection of the best among 35 configurations on only 200 validation campaigns.

D.2 Computational cost and performance of bidding policies

Table 10 reports the time to compute one bid, measured on the same 80 decision states of the main scenario ($\eta = 50$), on one CPU core of a laptop (Intel Core Ultra 7 265H) and of a cluster node (Intel Xeon E5-2695 v3); the time per campaign is the time per bid times 47.7 auctions. MPC's cost is linear in the number of rollouts M and in the horizon H (each rollout simulates the next H time units), whereas its reward saturates around $H = 25$ and $M = 40$–80; the best setting, MPC($H = 35$, $M = 60$, $\alpha_{\text{cont}} = 0.4$), costs about 2.3 s per bid on the laptop. Changing α_{cont} does not change the

cost. One prediction of the valuation head takes 0.13 ms, and the RL policies need less than 1 ms per bid.

Table 10: Decision time per bid (mean over 80 decision states, one CPU core) and mean episode reward at $\eta = 50$.

Policy	ms / bid (laptop)	ms / bid (cluster node)	s / campaign (laptop)	reward ($\eta = 50$)
FiPla	< 0.01	< 0.01	< 0.01	2.821 ± 0.160
Shading($\alpha = 0.4$, $M = 20$)	66.1	147	3.15	3.680 ± 0.234
Shading($\alpha = 0.4$, $M = 40$)	133	297	6.33	3.680 ± 0.234
Shading($\alpha = 0.4$, $M = 60$)	202	446	9.64	3.654 ± 0.232
MPC($H = 1$, $M = 20$)	155	308	7.40	3.458 ± 0.207
MPC($H = 1$, $M = 40$)	311	611	14.8	3.360 ± 0.193
MPC($H = 1$, $M = 100$)	769	1 538	36.7	3.309 ± 0.190
MPC($H = 1$, $M = 200$)	1 570	3 067	74.9	3.307 ± 0.190
MPC($H = 10$, $M = 20$)	373	878	17.8	3.911 ± 0.236
MPC($H = 10$, $M = 20$, $\alpha_{\text{cont}} = 0.4$)		same cost as $\alpha_{\text{cont}} = 1$		3.898 ± 0.239
MPC($H = 10$, $M = 40$)	734	1 745	35.0	3.862 ± 0.226
MPC($H = 10$, $M = 40$, $\alpha_{\text{cont}} = 0.4$)		same cost as $\alpha_{\text{cont}} = 1$		3.829 ± 0.228
MPC($H = 10$, $M = 60$)	1 120	2 615	53.4	3.938 ± 0.238
MPC($H = 25$, $M = 20$)	639	1 587	30.5	4.040 ± 0.256
MPC($H = 25$, $M = 20$, $\alpha_{\text{cont}} = 0.4$)		same cost as $\alpha_{\text{cont}} = 1$		4.120 ± 0.264
MPC($H = 25$, $M = 40$)	1 294	3 156	61.7	4.153 ± 0.262
MPC($H = 25$, $M = 40$, $\alpha_{\text{cont}} = 0.4$)		same cost as $\alpha_{\text{cont}} = 1$		4.228 ± 0.263
MPC($H = 25$, $M = 60$)	1 932	4 742	92.2	4.112 ± 0.257
MPC($H = 25$, $M = 60$, $\alpha_{\text{cont}} = 0.4$)		same cost as $\alpha_{\text{cont}} = 1$		4.285 ± 0.268
MPC($H = 25$, $M = 80$)	2 523	6 320	120	4.282 ± 0.268
MPC($H = 35$, $M = 20$)	781	1 950	37.2	4.030 ± 0.260
MPC($H = 35$, $M = 40$)	1 539	3 885	73.4	4.225 ± 0.273
MPC($H = 35$, $M = 60$)	2 278	5 859	109	4.206 ± 0.269
MPC($H = 35$, $M = 60$, $\alpha_{\text{cont}} = 0.4$)		same cost as $\alpha_{\text{cont}} = 1$		4.328 ± 0.275
PPO (inference)	0.574	0.739	0.0274	2.965 ± 0.182
SAC (inference)	0.441	0.697	0.0211	0.707 ± 0.119
TD3 (inference)	0.302	0.538	0.0144	2.006 ± 0.121

D.3 Robustness of MPC to valuation errors and fewer rollouts

We study how errors in the learned valuation head $\widehat{V}_0$ and the number of planning rollouts affect MPC's performance. Our reference is MPC($H = 25$, $M = 80$, $\alpha_{\text{cont}} = 1$), using the 32-32 ReLU head trained on 1 000 FiPla campaigns. We consider four types of changes (Table 11): (i) multiplicative noise, scaling each prediction by $e^{\sigma\varepsilon - \sigma^2/2}$, with $\varepsilon \sim \mathcal{N}(0,1)$ and $\sigma \in \{0.1, 0.25, 0.5, 1\}$; (ii) heads trained on only 10, 30, 100, or 300 campaigns; (iii) misspecified heads that ignore fatigue ($S = 1$), are trained with higher competition ($\text{LogNormal}(0,1)$), or have their predictions multiplied by 0.5 or 2; and (iv) fewer planning rollouts, keeping the reference head unchanged.

All variants are evaluated on the same 1 000 test campaigns, with paired differences and 95% confidence intervals. The valuation-head experiments also use the same rollout random numbers as the reference. Prediction errors are measured against the oracle V_0 on auctions with $V_0 > 0.05$.

Table 11: Performance of MPC under valuation errors and reduced rollout counts. The reference uses $H = 25$, $M = 80$ and the 32-32 ReLU valuation head trained on 1 000 campaigns. Correlation and mean absolute error are measured against the oracle V_0.

family	variant	corr.	$\lvert\widehat{V}_0 - V_0\rvert$	reward	Δ vs reference	Δ vs Shading
reference	exact head (1 000 campaigns), $M = 80$	0.97	0.050	4.282 ± 0.268	—	$+0.60 \pm 0.13$
noisy $\widehat{V}_0$	$\sigma = 0.1$	0.95	0.069	4.117 ± 0.260	-0.17 ± 0.10	$+0.44 \pm 0.12$
	$\sigma = 0.25$	0.88	0.125	4.118 ± 0.261	-0.16 ± 0.10	$+0.44 \pm 0.13$
	$\sigma = 0.5$	0.69	0.225	3.930 ± 0.251	-0.35 ± 0.10	$+0.25 \pm 0.11$
	$\sigma = 1$	0.34	0.455	3.579 ± 0.244	-0.70 ± 0.11	-0.10 ± 0.12
fewer training campaigns	10 campaigns	0.16	0.470	2.179 ± 0.218	-2.10 ± 0.22	-1.50 ± 0.21
	30 campaigns	0.62	0.265	3.774 ± 0.238	-0.51 ± 0.11	$+0.09 \pm 0.12$
	100 campaigns	0.75	0.212	4.195 ± 0.262	-0.09 ± 0.09	$+0.52 \pm 0.11$
	300 campaigns	0.94	0.082	4.250 ± 0.268	-0.03 ± 0.09	$+0.57 \pm 0.13$
misspecified head	ignores fatigue ($S = 1$)	0.76	0.167	3.799 ± 0.261	-0.48 ± 0.11	$+0.12 \pm 0.14$
	trained on higher competition	0.98	0.039	4.233 ± 0.267	-0.05 ± 0.08	$+0.55 \pm 0.12$
scaled $\widehat{V}_0$	$\widehat{V}_0 \times 0.5$	0.96	0.314	4.350 ± 0.293	$+0.07 \pm 0.10$	$+0.67 \pm 0.14$
	$\widehat{V}_0 \times 2$	0.97	0.492	3.827 ± 0.234	-0.45 ± 0.12	$+0.15 \pm 0.12$
planning rollouts	$M = 10$	—	—	3.881 ± 0.252	-0.40 ± 0.12	$+0.20 \pm 0.11$
	$M = 20$	—	—	4.040 ± 0.256	-0.24 ± 0.10	$+0.36 \pm 0.12$
	$M = 40$	—	—	4.153 ± 0.262	-0.13 ± 0.10	$+0.47 \pm 0.13$
	$M = 60$	—	—	4.112 ± 0.257	-0.17 ± 0.11	$+0.43 \pm 0.11$
baseline	Shading($\alpha = 0.4$, $M = 20$)	—	—	3.680 ± 0.234	-0.60 ± 0.13	—

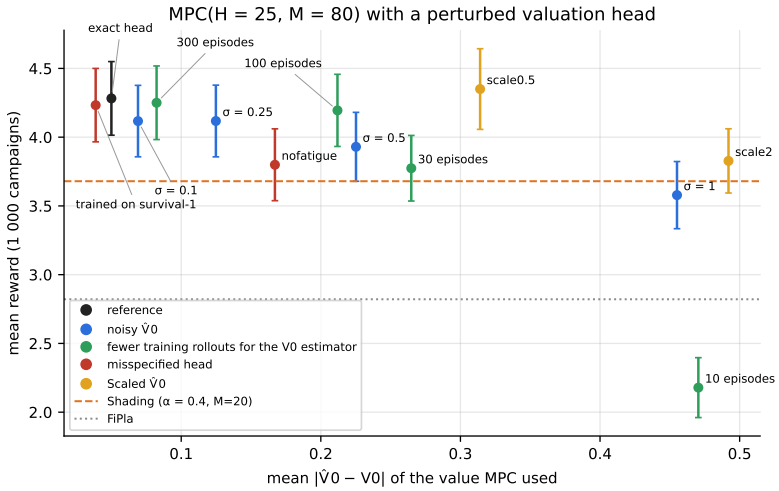


Figure 9: Mean episode reward of MPC($H = 25$, $M = 80$) as a function of the mean absolute error of its valuation head $\widehat{V}_0$, for the different perturbations.

MPC tolerates moderate valuation errors (Table 11, Figure 9). Heads trained on 100 or 300 campaigns perform close to the reference (-0.09 ± 0.09 and -0.03 ± 0.09). Multiplicative noise reduces the reward by about 0.17 for $\sigma \leq 0.25$ and 0.35 for $\sigma = 0.5$, but MPC still outperforms Shading. With $\sigma = 1$, it falls to Shading's level; with only 10 training campaigns, it performs substantially worse. Ignoring fatigue reduces the reward by 0.48 ± 0.11, bringing MPC close to Shading, whereas a head trained under higher competition performs similarly to the reference (-0.05 ± 0.08).

Errors in the scale of $\widehat{V}_0$ have asymmetric effects. Halving its predictions does not reduce performance ($+0.07 \pm 0.10$), as it gives a finer grid of candidate bids in the relevant range. Doubling them leads to excessive bids, including during rollout continuation, and reduces the reward by 0.45 ± 0.12. Reducing the number of rollouts has a smaller effect than large valuation errors: using $M = 20$–60 instead of 80 lowers the reward by 0.13–0.24, while even $M = 10$ remains above Shading ($+0.20 \pm 0.11$).

D.4 Evaluating bidding policies without delayed rewards

We evaluate a variant of the main scenario in which conversions occur immediately after a winning auction, with the same probability f as in the original scenario (Table 12). This removes delayed credit assignment while keeping the fatigue dynamics, competition, and campaign lengths unchanged. The variant, `survival-second-price-exponential-immediate`, also ensures that no conversion is lost after the last auction. MPC uses a valuation head trained on this scenario, while the RL agents follow the same hyperparameter search and training protocol as in the main benchmark.

Table 12: Comparison of the main scenario and its immediate-conversion variant. Only the timing and reporting of conversions change; the fatigue dynamics and conversion probabilities remain unchanged.

	Main scenario	Immediate rewards
Carry-over context	$S_u(t^-) = \{t_i < t\}$, the past wins	Same
Conversion probability of a won display	$f = \mathrm{clip}\big(\phi(S_u(t^-), t) \cdot (\mathbf{1}^\top u)(\mathbf{1}^\top x),\ 0, 1\big)$	Same f
Conversions per won display	At most 1	At most 1
Time of conversion	$t + s$, delay $s \sim \mathrm{Exp}(1)$	t (no delay)
Conversions after the last auction	Never credited (including after T)	Cannot occur
Reward received by the agent	Payment at the winning auction; conversion value at the first auction after $t + s$	Both at the winning auction
Oracle value V_0	$f(1 - e^{-(T-t)})$	f

Table 13: Performance of the bidding policies with immediate conversions. Results are mean episode rewards ($\pm$ 95% CI) over 1 000 test campaigns, with paired differences relative to MPC and Shading. For RL, the mean is taken over three trainings, whose individual test rewards are shown in parentheses.

Policy	reward	Δ vs MPC	Δ vs Shading
MPC($H = 25$, $M = 80$)	4.409 ± 0.258	—	$+0.59 \pm 0.13$
PPO (4.11 / 4.23 / 3.86)	4.069 ± 0.223	-0.34 ± 0.10	$+0.25 \pm 0.10$
Shading(best $\alpha = 0.4$, $M = 20$)	3.822 ± 0.223	-0.59 ± 0.13	—
FiPla	2.961 ± 0.161	-1.45 ± 0.18	-0.86 ± 0.15
GreedyMyopic	2.908 ± 0.155	-1.50 ± 0.18	-0.91 ± 0.15
TD3 (3.93 / 0.00 / 0.00)	1.309 ± 0.072	-3.10 ± 0.20	-2.51 ± 0.17
Constant(best $b = 0.11$)	1.092 ± 0.103	-3.32 ± 0.21	-2.73 ± 0.19

With immediate conversions, PPO outperforms all myopic baselines, including Shading ($+0.25 \pm 0.10$), but remains below MPC (-0.34 ± 0.10). Thus, removing delayed feedback does not eliminate the advantage of MPC, which can still plan around the effects of fatigue on future auctions. TD3 remains unstable: one training reaches 3.93, above Shading, while the other two converge to policies that never bid, as did 44 of its 96 hyperparameter configurations during the search.

Immediate conversions also prevent rewards from being lost after the last auction, unlike in the main scenario. Since the two scenarios do not share the same random draws, performance comparisons are made within each scenario.

D.5 Evaluating bidding policies against the optimal policy under saturation and recency

We consider two simpler scenarios in which the optimal bidding policy can be computed by dynamic programming (DP). Neither scenario has user or display features, and the carry-over state is one-dimensional. They differ in how past wins affect conversion probability:

- *Saturation:* $p(n) = e^{-n/10}$, where n is the number of past wins.
- *Recency:* $p(\tau) = 1 - e^{-0.5\tau}$, where τ is the time since the last win.

Both scenarios use exponentially distributed conversion delays with rate 0.5 and competing bids drawn from $\mathrm{LogNormal}(-1, 1)$.

We compare the bidding policies with the DP optimum on the same 1 000 test campaigns. Unless otherwise stated, MPC uses $M = 60$ rollouts and $\alpha_{\mathrm{cont}} = 1$, while the RL agents follow the training protocol of the main benchmark. We also evaluate PPO with the time since the last win added to its observations, and a diagnostic Cooldown policy that bids the oracle V_0 except during a fixed period after each win. The tables report the fraction of the improvement over FiPla achieved by each policy, defined as $(R - R_{\mathrm{FiPla}})/(R_{\mathrm{DP}} - R_{\mathrm{FiPla}})$, where R denotes the mean episode reward.

Computing the optimal policy. In both scenarios, the campaign duration is exponential with mean $\eta = 50$, giving a termination rate $h = 1/\eta = 0.02$. Auction arrivals have rate 1. Because the remaining campaign duration is memoryless, the optimal policy depends only on the current state s: the number of past wins n for saturation, or the time since the last win τ for recency. Before the first win, $\tau = \infty$ and $p(\tau) = 1$.

A display won in state s has expected conversion value

$$r(s) = p(s)\frac{\delta}{\delta + h},$$

where $\delta = 0.5$ is the conversion-delay rate. The factor $\delta/(\delta + h)$ accounts for the probability that the conversion occurs before the campaign ends.

The probability of reaching another auction before the campaign ends is $q = 1/(1 + h)$. Conditional on reaching it, the elapsed time is $G \sim \mathrm{Exp}(1 + h)$. A win changes the state to $s'_{\mathrm{win}} = n + 1$ in saturation or $s'_{\mathrm{win}} = G$ in recency; after a loss, the state is $s'_{\mathrm{lose}} = n$ or $s'_{\mathrm{lose}} = \tau + G$, respectively.

In a second-price auction, the optimal bid is the marginal value of winning:

$$b^*(s) = \max(D(s), 0), \qquad D(s) = r(s) + q\big(\mathbb{E}V(s'_{\mathrm{win}}) - \mathbb{E}V(s'_{\mathrm{lose}})\big),$$

where $V(s)$ is the optimal expected future reward and satisfies

$$V(s) = q\,\mathbb{E}V(s'_{\mathrm{lose}}) + \mathbb{E}\big[(D(s) - B)^+\big], \qquad B \sim \mathrm{LogNormal}(-1, 1).$$

The last expectation has a closed form. For saturation, we solve the Bellman equation by backward induction from $n = 200$, where conversion probability is negligible, using bisection to solve for each $V(n)$. For recency, we use value iteration on a grid $\tau \in [0, 60]$ with step 0.02, stopping when the maximum change falls below 10^{-9}.

The resulting policy bids $b^*(s)$ at every auction. Since it uses the known model, it provides an optimal reference in expectation, subject to numerical approximation and test-sample variability. Table 16 reports its effective shading multiplier $\alpha^*(s) = b^*(s)/r(s)$.

Why constant shading is close to optimal. In both scenarios, the optimal bid is approximately a fixed fraction of the myopic value $r(s)$ (Table 16). The effective multiplier $\alpha^*(s) = b^*(s)/r(s)$ ranges from 0.39 to 0.66 for the first 15 wins in saturation, and from 0.53 to 0.64 for the recency states considered. A single shading multiplier therefore approximates the optimal policy well. Indeed, Shading is statistically indistinguishable from the DP optimum in both scenarios: $+0.03 \pm 0.08$ in saturation and -0.09 ± 0.12 in recency.

With $M = 60$ rollouts, MPC remains below Shading in both scenarios. Increasing M to 1 000 substantially reduces this gap, particularly in recency, where MPC matches Shading ($+0.00 \pm 0.12$). This suggests that the accuracy of MPC's rollout estimates matters when the optimal multiplier varies little across states.

Table 14: Saturation kernel (RL: mean of the 3 trainings, the test reward of each in parentheses). MPC: $\alpha_{\text{cont}} = 1$ unless otherwise stated.

Policy	reward	Δ vs DP optimum	Δ vs Shading	share of DP gain
Shading(best $\alpha = 0.5$, $M = 20$)	4.351 ± 0.158	$+0.03 \pm 0.08$	—	105%
DP optimum	4.323 ± 0.162	—	-0.03 ± 0.08	100%
MPC($H = 25$, $M = 1\,000$)	4.237 ± 0.157	-0.09 ± 0.09	-0.11 ± 0.10	86%
PPO (4.20 / 4.08 / 4.24)	4.173 ± 0.143	-0.15 ± 0.07	-0.18 ± 0.08	75%
MPC($H = 25$, $M = 200$)	4.138 ± 0.150	-0.18 ± 0.10	-0.21 ± 0.10	69%
MPC($H = 25$, $M = 80$)	4.097 ± 0.154	-0.23 ± 0.11	-0.25 ± 0.10	62%
MPC($H = 10$, $M = 60$, $\alpha_{\text{cont}} = 0.5$)	4.097 ± 0.152	-0.23 ± 0.10	-0.25 ± 0.11	62%
MPC($H = 10$, $M = 60$)	4.056 ± 0.154	-0.27 ± 0.11	-0.30 ± 0.10	56%
MPC($H = 35$, $M = 60$)	4.051 ± 0.156	-0.27 ± 0.10	-0.30 ± 0.10	55%
MPC($H = 25$, $M = 60$, $\alpha_{\text{cont}} = 0.5$)	4.040 ± 0.148	-0.28 ± 0.10	-0.31 ± 0.10	53%
MPC($H = 35$, $M = 60$, $\alpha_{\text{cont}} = 0.5$)	4.039 ± 0.149	-0.28 ± 0.10	-0.31 ± 0.10	53%
MPC($H = 25$, $M = 60$)	4.029 ± 0.154	-0.29 ± 0.11	-0.32 ± 0.11	51%
Constant(best $b = 0.18$)	3.856 ± 0.169	-0.47 ± 0.12	-0.50 ± 0.12	22%
FiPla	3.721 ± 0.142	-0.60 ± 0.12	-0.63 ± 0.12	0%
GreedyMyopic	3.605 ± 0.151	-0.72 ± 0.13	-0.75 ± 0.12	-19%

Table 15: Recency kernel (RL: mean of the 3 trainings, the test reward of each in parentheses). MPC: $\alpha_{\text{cont}} = 1$ unless otherwise stated.

Policy	reward	Δ vs DP optimum	Δ vs Shading	share of DP gain
DP optimum	7.875 ± 0.529	—	$+0.09 \pm 0.12$	100%
MPC($H = 25$, $M = 1\,000$)	7.786 ± 0.521	-0.09 ± 0.12	$+0.00 \pm 0.12$	89%
Shading(best $\alpha = 0.6$, $M = 20$)	7.784 ± 0.520	-0.09 ± 0.12	—	88%
MPC($H = 25$, $M = 200$)	7.670 ± 0.516	-0.20 ± 0.13	-0.11 ± 0.13	74%
PPO + time since last win (7.66 / 7.63 / 7.63)	7.643 ± 0.510	-0.23 ± 0.10	-0.14 ± 0.10	70%
MPC($H = 25$, $M = 80$)	7.542 ± 0.510	-0.33 ± 0.13	-0.24 ± 0.14	57%
MPC($H = 10$, $M = 60$, $\alpha_{\text{cont}} = 0.6$)	7.498 ± 0.505	-0.38 ± 0.13	-0.29 ± 0.13	52%
MPC($H = 25$, $M = 60$, $\alpha_{\text{cont}} = 0.6$)	7.488 ± 0.504	-0.39 ± 0.13	-0.30 ± 0.13	50%
MPC($H = 35$, $M = 60$, $\alpha_{\text{cont}} = 0.6$)	7.488 ± 0.504	-0.39 ± 0.13	-0.30 ± 0.13	50%
MPC($H = 25$, $M = 60$)	7.483 ± 0.505	-0.39 ± 0.14	-0.30 ± 0.13	50%
MPC($H = 35$, $M = 60$)	7.483 ± 0.505	-0.39 ± 0.14	-0.30 ± 0.13	50%
MPC($H = 10$, $M = 60$)	7.476 ± 0.505	-0.40 ± 0.14	-0.31 ± 0.13	49%
Constant(best $b = 0.31$)	7.099 ± 0.489	-0.78 ± 0.15	-0.68 ± 0.15	1%
FiPla	7.095 ± 0.490	-0.78 ± 0.18	-0.69 ± 0.17	0%
PPO (7.09 / 7.02 / 7.03)	7.048 ± 0.464	-0.83 ± 0.14	-0.74 ± 0.13	-6%
Cooldown(oracle V_0, best $\tau = 1$)	6.998 ± 0.473	-0.88 ± 0.17	-0.79 ± 0.15	-12%
GreedyMyopic	6.856 ± 0.478	-1.02 ± 0.18	-0.93 ± 0.16	-31%

Table 16: Optimal effective shading $\alpha^* = b^*/r$ of the DP by state (n: past wins; τ: time since the last win).

saturation, n	0	2	5	10	15	20
$\alpha^* = b^*/r$	0.39	0.42	0.46	0.55	0.66	0.76
recency, τ	0.25	1	2	3	5	10
$\alpha^* = b^*/r$	0.64	0.60	0.57	0.55	0.54	0.53

Effect of state representation on PPO. PPO performs close to Shading in saturation, where the number of past wins is directly available in its observation. In recency, however, PPO receives the current time and the time of the last win as separate inputs (with -1000 indicating no previous win). Its bids barely depend on their difference, the time elapsed since the last win, and its performance is close to that of a constant bidder. Adding this elapsed time explicitly to the observation, while keeping the same hyperparameters and training seeds, improves PPO's reward by about 0.6 and brings it close to Shading (-0.14 ± 0.10). The gap in this scenario is therefore largely explained by the state representation.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: it does

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: it does

   Guidelines:

   - The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
   - The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
   - If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
   - While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: [Yes]

Justification: it does

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

   Answer: [Yes]

   Justification: it does

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
   - If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
   - Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
   - While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
     (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
     (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
     (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
     (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: it does

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: [Yes]

   Justification: it does

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
   - The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: [Yes]

   Justification: it does

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
   - The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
   - The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
   - The assumptions made should be given (e.g., Normally distributed errors).
   - It should be clear whether the error bar is the standard deviation or the standard error of the mean.

- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [Yes]

   Justification: it does

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
   - The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: [Yes]

   Justification: we reviewed it

   Guidelines:

   - The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
   - If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
   - The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: [Yes]

    Justification: help practitioners and theoreticians to better train RL model on repeated auctions with dynamic value.

    Guidelines:

    - The answer [N/A] means that there is no societal impact of the work performed.
    - If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
    - Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.

- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

    Answer: [N/A]

    Justification: no release of pre trained LLM or data.

    Guidelines:

    - The answer [N/A] means that the paper poses no such risks.
    - Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
    - Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
    - We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

    Answer: [Yes]

    Justification: we are the creator of this simulator

    Guidelines:

    - The answer [N/A] means that the paper does not use existing assets.
    - The authors should cite the original paper that produced the code package or dataset.
    - The authors should state which version of the asset is used and, if possible, include a URL.
    - The name of the license (e.g., CC-BY 4.0) should be included for each asset.
    - For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
    - If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
    - For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.

- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: [Yes]

    Justification: it is the case

    Guidelines:

    - The answer [N/A] means that the paper does not release new assets.
    - Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
    - The paper should discuss whether and how consent was obtained from people whose asset is used.
    - At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: [N/A]

    Justification:

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
    - According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: [N/A]

    Justification:

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
    - We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
    - For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [N/A]

Justification: only used for writing and code cleaning purpose.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.